

UNIDAD 1: INGENIERIA DE SOFTWARE EN CONTEXTO

QUE ES EL SOFTWARE

- Conjunto de programas con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente (archivos de configuración usados para la ejecución, documentos para el usuario, documentos que describen la estructura del sistema) junto con las herramientas usadas para su construcción
- No es solo código, es el programa + la documentación (va desde los requerimientos, algo abstracto, hasta cosas más detalladas como el código que se ejecuta)
- No es suficiente saber programar para hacer software

TIPOS BÁSICOS DE SOFTWARE

- **Software del sistema software**
 - o Controla y gestiona los recursos de hw de una computadora y proporciona una interfaz de usuario
- **Utilitarios**
 - o Subconjunto de software de sistema que proporciona utilidades o herramientas adicionales para mejorar y optimizar el rendimiento de SO y del hardware de la computadora
- **Software de aplicación**
 - o Realizar tareas específicas para satisfacer las necesidades de los usuarios (procesar texto, crear presentaciones, navegar internet, etc), es con el que vamos a trabajar

SOFTWARE Y MANUFACTURA

No se puede comparar software con una línea de producción, existen razones que muestran la intangibilidad del software como producto

- El software es menos predecible, no siempre vamos a tener los mismos resultados
- Casi ningún producto de software es igual a otro, ya que los requerimientos o necesidades de los usuarios que van a usar el software van a ser distintas por más que el concepto sea el mismo
- No todas las fallas en software son errores (tratar errores en software es distinto que en producción)
- El software no se gasta (puede ser que tenga que adaptarse al cambio por problemas de vigencia, pero no se gasta), los productos generalmente se desgastan con el paso del tiempo
- El software no está gobernado por las leyes de la física

PROBLEMAS AL DESAROLLAR / CONSTRUIR SOFTWARE

- Más tiempos y costos que los presupuestados
- Versión final del producto que no satisfaga las necesidades del cliente (puede pasar que no le satisfaga pero que efectivamente sea lo que el pidió)

- **Problema en la escalabilidad y/o adaptabilidad del software**
 - o Agregar funcionalidades en otras versiones es una misión imposible
- **Mala documentación**
 - o Documentación desactualizada que no acompaña al software, trayendo inconsistencias en la gestión o mantenimiento del mismo
- **Mala calidad de software**
 - o Muchas veces el software de calidad se lo relaciona con la cantidad de testing que se realizó sobre él, pero esto no es así

INGENIERÍA DE SOFTWARE

- Disciplina que se encarga de todos los aspectos y etapas de la producción del software, desde la primera etapa hasta la puesta en producción(desarrollo, validación, mantenimiento, evolución, etc)
- **Es importante ya que**
 - o Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables
 - o A largo plazo es más barato usar métodos y técnicas de ingeniería de software para los sistemas de software (la mayoría de los costos consisten en cambiar el software después de que se ponen en producción)
- **La ingeniería de software se apoya en varias realidades**
 - o Se debe hacer un esfuerzo para entender el problema y las necesidades del cliente antes de desarrollar una aplicación de software
 - o El diseño es una actividad crucial en el desarrollo de software y hacerla correctamente brinda software de calidad y fácil de mantener
- Nace tras la crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente a la baja eficacia que presentan una gran cantidad de empresas al momento de producir software

CASOS FALLIDOS Y EXITOSOS

- **Fallidos**
 - o Ubuntu pone – FBI virtual case file
- **Exitosos**
 - o Linux – amazon - android

DISCIPLINAS DE SOFTWARE

Se puede dividir en tres grandes ramas o disciplinas

- **Disciplinas técnicas** → actividades que aportan al desarrollo de software como producto
 - o Testing – construcción – despliegue – análisis y diseño – requerimientos – implementación - documentación
- **Disciplinas de gestión** → Planificación, monitoreo y control de proyectos de desarrollo de software
 - o Metodologías Lean agiles
- **Disciplinas de soporte** → disciplinas transversales a los procesos de software, permiten verificar la integridad y calidad de un producto e software
 - o Gestión de configuración de software – calidad - métricas

PROCESO DE SOFTWARE

- **Proceso** → secuencia de pasos ejecutados para un propósito dado
- **Proceso de software** → conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados
- Es un conjunto estructurado de actividades que tienen como objetivo, mediante un conjunto de entradas, obtener como salida un producto de software o un servicio de calidad q aporte valor
- Lo define la organización en función de los objetivos y características del software
- **Las entradas pueden ser**
 - o Requerimientos (definen límites al alcance del producto)
 - o Personas
 - o Otros: materiales, energía, equipamiento, procedimientos, librerías, hardware, etc
- **Características**
 - o Son complejos, es por eso que no existe uno ideal; además, la mayoría de las organizaciones diseñan sus propios procesos de desarrollo de software
 - o Han evolucionado con el tiempo para beneficiarse de las capacidades de la gente en una organización y de las características específicas de los sistemas que se están desarrollando (algunos sistemas, como los críticos, tienen desarrollo muy estructurado – proceso definido- mientras que otros, como los empresariales, tienen requerimientos muy cambiantes – es más efectivo un proceso menos formal y flexible, usando filosofía ágil -
 - o Debe ser explícitamente modelado si va a ser administrado
 - o Las actividades varían según el tipo de organización y el sistema a desarrollar (análisis – diseño – prueba – etc) pero siempre deben incluir productos, roles, responsabilidades y condiciones

FACTORES DEL PROCESO DE SOFTWARE

- **Procedimientos y métodos**
 - o Adaptar los procesos establecidos según el contexto (definir qué hacer y que no hacer)
 - o Es importante documentar esta información
- **Personas**
 - o Deben estar motivadas, capacitadas y contar con habilidades
 - o Es el principal capital del desarrollo del software y es la parte más cara; ya que el desarrollo es una actividad humano intensiva, las personas son las que hacen el software y sin ellas no podemos construir nada
- **Herramientas y equipos**
 - o Materiales necesarios para llevar a cabo el proceso (permiten transformar entradas en salidas)
 - o Se recomienda automatizar la mayor cantidad de actividades posibles, para mejorar la eficiencia de las personas y centrar el foco de atención de las personas en otras cosas

PROCESOS DEFINIDOS

- Inspirado en las líneas de producción
- Tengo un conjunto de entradas que se utilizan en un conjunto de actividades, las cuales van a dar siempre el mismo resultado si mis entradas son las mismas (se asume que podemos repetir el proceso indefinidamente y obtener los mismos resultados)
- Esa repetibilidad se usa para poder predecir que va a pasar y poder tener una administración y control del proceso definido (ilusión de control que claramente es falsa en el software)
- Puedo usar cualquiera de los ciclos de vida
- Un ejemplo de esto es el PUD

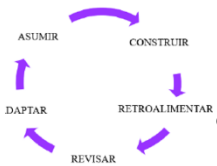
El principio de 'mismas entradas → mismas salidas' es poco aplicable al desarrollo de software porque no es un proceso determinista: intervienen factores humanos, decisiones de diseño y cambios de contexto que hacen que, aun con condiciones similares, los resultados no sean idénticos

PROCESOS EMPIRICOS

- Se basan en la experiencia del usuario y del negocio (no siempre fácil de conseguir)
- Asume un proceso complejo en el que existen variables cambiantes que no pueden repetirse ni controlarse en su totalidad, repetir el proceso puede implicar resultados distintos por lo que debemos adaptarnos para poder garantizar software de calidad
- No se cubre la totalidad de cosas a hacer, eso se deja librado para que cada equipo pueda decidir ciertas cosas basándose en la experiencia (no es un proceso completo)
- Trabajan bien con procesos creativos y complejos
- La administración y control es mediante inspecciones frecuentes y adaptaciones
- Son los que usamos para lean y agile
- Aprendemos de la experiencia mediante ciclos de retroalimentación que permiten que mejoremos
- Se suele recomendar trabajar con el modelo iterativo-incremental y prohibir el secuencial

- Patrón de conocimiento de procesos empíricos

- Es un proceso iterativo donde se parte de una **hipótesis (asumir)**, luego se **construye** una solución, se miden y analizan resultados, y a partir de esa información se realiza una **retroalimentación**
- Con esos datos, se **revisa** lo obtenido y se **adapta** el proceso según los resultados (si algo sale mal se lo adapta, ajustando la hipótesis inicial y se arranca de nuevo)
- Este ciclo se repite continuamente, lo que permite mejorar progresivamente el producto o proceso. Por eso, se lo considera un enfoque iterativo e incremental, donde el aprendizaje surge de la experiencia y la mejora constante.
- Esta experiencia se aplica únicamente al mismo equipo NO PUEDE extrapolarse a otros equipos (ósea si en x equipo algo funcionó de x manera, no significa que en otro proyecto funcionará igual, se debe adaptar a cada contexto en especial)



CICLOS DE VIDA

- Serie de pasos a través de los cuales un producto o proyecto progresa en el tiempo
- Es una abstracción que define cuáles son las etapas o fases del proceso (requerimientos, especificaciones, diseño, etc) y el orden de cada una de ellas (no especifica que hacer ni cómo ni con que) y establecen los criterios para saber cuándo pasar de una tarea a la siguiente
- **Ciclo de vida de un proyecto**
 - Es una representación de un proceso
 - Grafica la descripción del proceso desde una perspectiva particular
 - Define las fases del proceso y el orden en el cual se van a llevar a cabo
 - Su ciclo de vida dura lo que dura el desarrollo del software
- **Ciclo de vida de un producto**
 - Inicia con la idea de crear un producto de software y termina cuando a ese producto se lo saca del mercado (puede durar años, por lo tanto es mayor al ciclo de vida del proyecto)
 - Cuando existen nuevas necesidades de evolución (cambios/actualizaciones) se crea otro proyecto distinto para generar otra versión de ese producto (un ciclo de vida de un producto puede haber n ciclos de vida de proyectos)
 - Si a un producto en el mercado no le surgen cambios significa que el producto no se está usando

CICLOS DE VIDA BÁSICOS

- **Secuencial**
 - Las actividades se ejecutan de forma lineal, el proyecto progresa a través de una secuencia ordenada de pasos en la que una actividad no puede iniciar sin que la anterior haya finalizado
 - El avance entre fases se da tras una revisión final de cada una de estas para determinar si el software está listo para avanzar
 - Usado en sistemas con alta documentación, donde los requerimientos son exhaustivos y están bien comprendidos (baja incertidumbre)
 - Un ejemplo es el modelo en cascada

PROS	CONTRAS
Se minimizan gastos generales de planificación ya que la misma se puede realizar toda por adelantado	Los proyectos raramente tienen flujo secuencial
Muy funcional en productos con definiciones claras	Se necesitan requerimientos muy explícitos desde un principio y eso no suele suceder
La documentación generada a lo largo del proceso proporciona indicaciones significativas del progreso a lo largo del ciclo de vida	Se obtienen versiones funcionales en etapas muy avanzadas del proyecto (no hay entregas parciales)
	Encontrar un defecto implica un gran rediseño de la solución
	Un equipo de trabajo no puede comenzar hasta que otro no haya terminado
	Documentación burocrática y excesiva

- Iterativo / incremental

- A medida que avanza el calendario de actividades, se aplican iteraciones sucesivas de manera escalonada. C/u produce un incremento de software funcional potencialmente entregable
- Cada iteración permite adaptar y mejorar el proceso y mejorar la experiencia del equipo
- En cada incremento se añaden funcionalidades a la versión anterior (por lo general, los primeros incrementos incluyen funciones básicas que requiere el cliente)
- Vinculado con validación, especificación y desarrollo
- Es el ciclo de vida usado en metodologías ágiles

PROS	CONTRAS
Se reduce el costo de adaptar los requerimientos cambiantes del cliente	Se invisibiliza el proceso
Menor trabajo ya que en cada incremento se obtiene una retroalimentación que permite adaptar el modelo de las necesidades del cliente	La estructura de software tiende a degradarse frente a una cantidad considerable de incrementos
El cliente es parte del proceso de desarrollo	Alto costo en documentar cada incremento
Los clientes pueden usar y ganar valor del software en etapas muy tempranas (entregas parciales)	Existe incertidumbre
Útil en sistemas con requerimientos cambiantes	

- Recursivo

- Permite gestionar los riesgos del desarrollo de sistemas complejos a gran escala
- Requiere la intervención del cliente
- Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno de ellos, los riesgos más relevantes hasta que no quede ninguno
- Un ejemplo es el modelo en espiral, que cuenta con 4 fases
 - Determinar objetivos → identificar y resolver riesgos → desarrollar y probar → planificar la próxima iteración
- Esta en desuso ya que se ha comprobado que no son útiles y no funcionan
 - Mas costoso en tiempo y dinero readaptarlos para usarlos en otros proyectos
 - Puede carecer de documentación, dificultando mucho las tareas
 - Se genera una versión del producto recién al final
 - No genero varias versiones del producto (muestro prototipos o secciones no productivas)

ADMINISTRACIÓN DE PROYECTOS DEPENDIENDO DEL CICLO DE VIDA

- Durante el desarrollo del proyecto, la gestión del mismo se hace en base al ciclo de vida elegido (no es lo mismo gestionar un proyecto iterativo que uno secuencial) es por eso que el ciclo de vida elegido afecta directamente a la administración del proyecto
- Por ese motivo es importante elegir el ciclo de vida a implementar que mejor se alinee con las características del proyecto y producto a construir (evaluar riesgo, claridad y cantidad de requerimientos, conocimientos técnicos del equipo de desarrollo, plazos para ejecutar el proyecto y la posibilidad de validar los resultados del trabajo con el cliente)

El PROCESO es una implementación del CICLO DE VIDA (que es una abstracción que nos guía en cuáles son las etapas y su orden) que tiene un objetivo que lo guía, que es la creación de un PRODUCTO (debe ser no ambiguo y alcanzable)

CRISIS DEL SOFTWARE

- Conjunto de hechos relativos al software, que planteó Bauer en la conferencia de la OTAN en 1968, recalcando la dificultad para producir software libre de defectos, fácilmente comprensible y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra
- Causas:
 - o La evolución de la tecnología del hardware mediante los circuitos integrados permitió la posibilidad del desarrollo de sistemas más grandes y complejos (el salto en el hardware no fue acompañado por un salto en el desarrollo del software)
 - o La demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas
- Actualidad
 - o Muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:
 - o Demandas crecientes:
 - A medida que la tecnología y las técnicas de desarrollo evolucionan, la demanda de sistemas más grandes y complejos aumenta.
 - Los usuarios necesitan que se construyan y distribuyan de manera rápida, y los métodos existentes de ingeniería no permiten lograr esta “espontaneidad”, por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.

- Bajas expectativas
 - Muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo que el software con frecuencia es más costoso y menos confiable de lo que debiera.
 - La consecuencia, recae en la conformidad de las empresas y de los usuarios ante un software. Las empresas se conforman con que el software solo “funcione”, sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario.

CUERPO DE CONOCIMIENTOS DE LA INGENIERIA DE SW (SEBOK)

- **Están conformado por 15 áreas de conocimiento**
 - Requerimientos de software
 - Diseño de software
 - Construcción de software
 - Testing de software
 - Mantenimiento de software
 - Gestión de la configuración de software
 - Gestión de la ingeniería de software
 - Proceso de la ingeniería de software
 - Modelos y métodos de la ingeniería de software
 - Calidad del software
 - Práctica profesional de la ingeniería del software
 - Economía de la ingeniería de software
 - Fundamentos matemáticos, científicos e ingenieriles

UNIDAD 2: GESTIÓN DE CONFIGURACIÓN DE SOFTWARET (SCM)

INTRODUCCIÓN

- Cuando hablamos de software no es solo el producto, sino todo el entorno (programas, procedimientos, reglas, documentación, datos, etc)
- Los sistemas de software cambian mucho durante su desarrollo y uso (cambios del negocio, nuevos requerimientos, reorganización de prioridades, cambios en el presupuesto, etc)
- Dichos cambios crean nuevas versiones del sistema y hacen que el software evolucione
- Los sistemas pueden verse como un conjunto de versiones donde c/u de ellas debe mantenerse y gestionarse para no perder la trazabilidad de los cambios que se incorporan a cada versión, es por eso que utilizamos SCM para mantener la integridad del producto de software

QUE ES SCM

- Disciplina de soporte transversal a todo el proyecto (es una actividad “paraguas”)
- Aplica dirección y monitoreo administrativo y técnico con el propósito es resolver distintos problemas, manteniendo la integridad del **producto** de software a lo largo de todo su ciclo de vida
- Su propósito no es controlar cambios y versiones, es MANTENER LA INTEGRIDAD
- **Algunos de estos problemas con el manejo de componentes son**
 - o Pérdida de componentes
 - o Superposición de cambios y/o pérdida de cambios (usar una versión q no es la ultima)
 - o Cambios no validos
 - o Regresión de fallas
 - o Doble mantenimiento
 - o Sincronía fuente – objeto- ejecutable
- **Algunas cuestiones que monitorea y administra para mantener la integridad de los productos**
 - o Identificar, documentar, controlar, registrar y reportar cambios en características técnicas y funcionales de los ítems de configuración
 - o Verificar que exista una trazabilidad con los requerimientos
 - o Mantener la integridad y origen
- Va a existir SCM mientras que el producto exista; esta tarea es responsabilidad de todo el equipo (aunque existen roles específicos que tienen tareas adicionales, como el gestor de configuración)

CUAL ES EL PROBLEMA QUE SE ESTÁ TRATANDO DE SOLUCIONAR

- El problema es que el software se puede modificar muy fácilmente, con entrar a un repositorio y hacer un click se pueden perder meses de trabajo
- Lo que se busca con SCM es evitar estas cosas, buscando que el producto sea íntegro y tenga un control de cambios en cada momento (que cada ítem de configuración tenga su versión)
- La SCM ayuda a que el resto de las disciplinas funcionen y se asegure la calidad del software
 - o Control de calidad de procesos
 - o Control de calidad de producto
 - o Prueba de software

QUE ES LA INTEGRIDAD

- Medio por el cual garantizamos que el producto a entregar tiene una calidad correspondiente, nos garantiza un nivel mínimo de confiabilidad
- El problema es que la calidad es subjetiva y el software intangible, lo que hace difícil medir si la calidad cumple con las expectativas del cliente
- **Se mantiene la integridad de un producto de software cuando**
 - o Satisface las necesidades del usuario (hace lo que el usuario espera y cumple requerimientos)
 - o Satisface criterios de performance y RNF
 - o Cumple con expectativas de costo (tanto del equipo como del cliente)
 - o Permite la rastreabilidad durante todo su ciclo de vida
 - Existen conexiones entre los ítems de configuración (definidos en el primer ciclo de vida del producto) que permiten analizar donde impacta un cambio de un ítem
 - De esta forma se puede determinar como impactará cada cambio de requerimientos mediante la trazabilidad
 - Más vínculos = mas información = mas trazabilidad = más costo

CONCEPTOS GENERALES

- ITEM DE CONFIGURACIÓN

- o Artefactos que forman parte del producto o proyecto y pueden almacenarse en un repositorio sin importar su extensión/tipo
- o Pueden sufrir cambios o necesitan ser compartidos entre los miembros equipo; y se desea conocer su estado y evolución a lo largo del ciclo de vida
- o Es de cualquier aspecto asociado al producto de software que se desea mantener (requerimientos, diseño, código fuente y/o ejecutable, datos de prueba, documentos, etc)
- o Pueden durar un ciclo de vida del producto, de producto o un sprint
- o Los define el equipo del proyecto

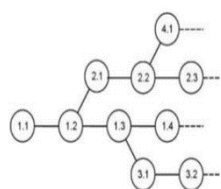
- VERSIÓN

- o Instancia de un ítem de configuración que difiere de otras instancias de ese ítem y se puede identificar unívocamente
- o Es un punto particular en el tiempo de un ítem de configuración (como un estado)
- o Desde el punto de vista de la evolución es la forma particular de un artefacto en un instante o contexto dado
- o El control de versiones se refiere a la evolución de un único ítem de configuración o de cada ítem por separado. Esa evolución lineal de un ítem puede representarse como



- VARIANTE

- o Versión de un ítem de configuración (o de la configuración) que evoluciona por separado
- o Representan configuraciones alternativas
- o Si la diferencia entre las instancias de un mismo ítem de configuración son muy pequeñas, no se denomina versión, sino variante
- o Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale. Las variantes ayudan a resolver conflictos como por ejemplo compatibilidad de un producto con diferente hardware o SO



- CONFIGURACIÓN DE SOFTWARE

- Conjunto de todos los ítems de configuración con su versión específica en un momento determinado (es una foto de todos los ítems en un momento específico, junto a su versión)
- Conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar

- REPOSITORIO

- Es un contenedor de ítems de configuración que se encarga de mantener la historia de cada ítem junto a sus atributos y relaciones
- Además es usado para hacer evaluaciones de impacto de los cambios propuestos (si me entra un cambio evalúo el impacto que va a generar, y cuando dinero y tiempo me tomará)
- Puede ser una o varias bases de datos
- Tiene una estructura que se debe respetar para mantener un orden y tener integridad, esto ayuda también a la seguridad, controles de acceso, políticas de backup (obviamente tener todo ordenado me ayuda a encontrar más fácilmente lo que busco)

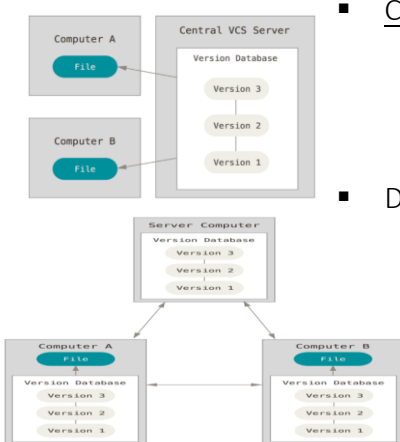
○ Tipos

■ Centralizados

- Un servidor contiene todos los archivos y sus versiones
- Los administradores tienen un mayor control sobre el repositorio
- Si el servidor falla estamos al horno

■ Descentralizados

- Cada cliente tiene una copia exactamente igual del repositorio
- Se resuelve el problema de los centralizados, si el servidor falla, solo es cuestión de “copiar y pegar”
- Posibilita otros workflows no disponibles en el modelo centralizado
- Es más complicado llevar un control sobre el mismo



- Release

- Entrega de un sistema que se libera para su uso los clientes u otros usuarios de la organización

- Rama (Branch)

- Es un conjunto de ítems de configuración con sus correspondientes versiones
- Existe una rama principal (trunk, master) que es por lo general la versión estable
- Nos permiten bifurcar el desarrollo de software, algunos de los motivos son:
 - Creación con semántica
 - Experimentación
 - Resolución de errores en desarrollo
 - Desarrollar un mismo software para diferentes plataformas
- No es una buena práctica trabajar sobre la rama principal
- Integrar ramas (merge)
 - Es el proceso de llevar los cambios realizados en ramas a la rama principal
 - Pueden surgir conflictos que hay que resolver (diff)
 - Todas las ramas deberían integrarse eventualmente a la principal o ser descartadas

- LÍNEA BASE

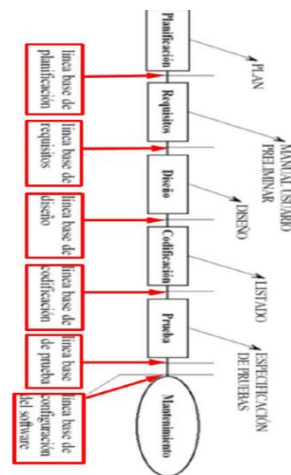
- Es una configuración de software (conjunto de ítems de configuración) que ha sido formalmente revisada y aprobada, sobre la que se ha llegado a un acuerdo
- Sirve como base para desarrollos posteriores y para tener un punto de referencia. Me permite hacer roll-backs (se cual es ultima situación estable en un momento de tiempo determinado, por lo que si es necesario volver atrás en, sé que ese es un punto estable y revisado)
- Los ítems de configuración, generalmente, se pueden modificar sin problema, pero cuando pertenecen a una línea base solo puede cambiarse mediante un protocolo formal de control de cambios (existe un procedimiento a seguir, y en caso de aceptarse se deben acordar nuevamente los parámetros a usar y comunicar cambios al team)
- Debe marcarse en un repositorio para tener un punto de referencia y no confundirse con la versión del producto (lo hago mediante tags). Marco solo los ítems específicos, no toda la configuración del software
- Debemos acordar parámetros para determinar que debe cumplir un ítem para considerarse línea base
- Pueden ser

- **De especificación**

- Son las primeras líneas base, no cuentan con código
- Son de requerimientos, diseño

- **De productos**

- Contiene una versión del producto cuyo código es ejecutable
- Han pasado un control de calidad definido previamente
- La primera línea base corresponde con el primer reléase
- Sirven para prueba o producción



ACTIVIDADES FUNDAMENTALES DE LA GESTIÓN DE CONFIGURACIÓN DE SOFTWARE

- Estas actividades son las cosas que contienen las secciones de un plan de gestión de configuración (recordar que en la gestión de configuración se debe plantificar)
- Todas las actividades se realizan en ambas metodologías (tradicional y ágil), menos la auditoria (ya que ser auditado por alguien externo no es compatible con una metodología ágil pura)
- **Identificación de ítems de configuración**
 - Identificación unívoca para cada ítem (así como la duración de su integridad, que puede diferir en función del tiempo que sea necesario mantenerlos)
 - El equipo define políticas y reglas de nombrado y versionado
 - Se debe definir la estructura del repositorio con un nombre representativa y la ubicación de cada uno de los ítems de configuración dentro de esa estructura
 - Debemos vincular los ítems de configuración con el lugar físico del repositorio donde se van a almacenar.
 - Se clasifican en

Ítems de producto	Ítems de proyecto	Ítems de iteración
	Es el plan del proyecto, el listado de defectos encontrados	Una vez terminada la iteración ese ítem deja de ser relevante (no significa que se borra, sino que no sirve después de eso)
Ciclos de vida más largo, se mantiene mientras el producto exista	Tienen un ciclo de vida de proyecto (duración definida, que impacta en el esquema de nombrado)	Es el ciclo de vida más corto
<u>Ejemplos:</u> Documento de arquitectura, casos de uso y de prueba, código, manual de usuario, BSD, ERS	<u>Ejemplos:</u> Plan de proyecto, cronograma, ítems a nivel proyecto	<u>Ejemplos:</u> Plan de iteración, reporte de defectos, cronogramas de iteración

- Control de cambios

- Una vez definida una línea base, no se la puede cambiar sin pasar por un proceso de control de cambios sobre los ítems que pertenecen a esa línea base (controlamos la línea base, ya que todos los trabajadores de software tienen los ítems de dicha base línea como referencia)
- Permite tener una trazabilidad entre los ítems de configuración (ante un cambio saber cuáles ítems están afectados)
- Involucra distintos actores y una evaluación del impacto del cambio
- Comité de control de cambios
 - El proceso lo lleva a cabo el comité de control de cambios,
 - Se reúne para autorizar la creación y cambios sobre la línea base
 - Está compuesto por los representantes de todas las áreas involucradas en el desarrollo (la integración del comité dependen de la propuesta del cambio)
 - Áreas → análisis y diseño – implementación – testing – arquitectura – desarrollo – líder del proyecto (y cualquier otro interesado en evaluar y enterarse del cambio)
- Etapas del proceso para gestionar una propuesta de cambio sobre una línea base
 - Se recibe una propuesta de cambio sobre una línea base (no sobre un ítem)
 - El comité realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto de la gestión de recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto
 - Si se autoriza la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios a revisar y auditar
 - Una vez realizado el cambio, el comité aprueba la modificación de la línea base y la marca como línea base nuevamente (revisión de partes)
 - Se notifica a los interesados de los cambios realizados sobre la línea base

- Auditorías de configuración de software **(NO PUEDO HACERLA EN ÁGIL)**

- Controles autorizados a realizar por el equipo de desarrollo en un momento determinado
- Se realiza mientras se construye el producto y tiene como objetivo tener calidad e integridad
- Una persona externa al equipo (auditor) que es independiente y objetiva, se encarga de analizar **líneas base** (el auditor puede detectar defectos y ajustar correcciones)
- Me permite analizar en un momento determinado el estado del software y ver si se cumplen los requerimientos, la intención de la línea base anterior y si se está siguiendo la planificación especificada en el plan de gestión de configuración

- Sin plan ni línea base, no se puede auditar
- Es una actividad
 - **Objetiva** (hay independencia de criterio, el auditor no está condicionado)
 - **Independiente** (no depende jerárquica, funcional ni salarial del equipo/elemento a auditar)
- Sirve a dos procesos básicos: validación y verificación
 - **Validación**
 - Se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, para que el cliente obtenga el producto correcto, lo que necesita
 - **Verificación**
 - Asegurarse que un producto cumple con los objetivos definidos para cada ítem de configuración, en la documentación de líneas base (todas las funciones son llevadas a cabo con éxito y los test cases dan “ok” o consten como problemas reportados en la nota del relese)
- Tipos de auditorias
 - Auditoria física de configuración/contingencia (PCA)
 - Es la que primero se hace y se encarga de **VERIFICAR** (si esta no pasa, la auditoria funcional no se realiza)
 - Asegura que lo que está indicando para cada ítem de configuración en la línea base o en la actualización se ha alcanzado realmente (verifica que la documentación a entregar es consistente con el producto construido)
 - Valida que el repo este donde debe estar, que los IC respeten esquema de nombrado, que estén guardados en donde deberían, etc
 - Auditoria funcional de configuración (FCA)
 - Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de los requerimientos
 - **VALIDA** que cada ítem de configuración sea consistente con la ERS
 - No puede realizarse sin una auditoria física previa que haya pasado

- Informes de estado

- Se ocupa de mantener los registros de la evolución del sistema y dónde está ubicado el software comprado con lo que está en la línea base
- Incluyen todos los cambios realizados a las líneas base durante el ciclo de vida del software, lo que normalmente es mucha información y salidas (por lo que se suele implementar en procesos automatizados)
- El objetivo principal es asegurarse de que la información de los cambios lleguen a todos los involucrados (evitar que el equipo trabaje en versiones anteriores, generando perdidas)
- Algunas preguntas que se pueden responder
 - ¿Cuál es el estado del ítem?
 - ¿Un requerimiento de cambio ha sido aprobado o rechazado por el comité?
 - ¿Qué versión de ítem implementa un requerimiento de cambio aprobado?
 - ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

PLAN DE GESTIÓN DE CONFIGURACIÓN

- El plan de configuración se define al principio y debería incluir respuestas a las preguntas de cómo se hacen las 4 actividades de la gestión de la configuración:
 - o Reglas de nombrado de los CI
 - o Herramientas a utilizar para SCM
 - o Roles e integrantes del comité
 - o Procedimiento formal de cambios
 - o Plantillas de formularios
 - o Procesos de auditorías
 - o Estructura del repositorio
 - o Cantidad, tipo y momento de auditoría
 - o Tipo de documentos

SCM EN AGILE

- En metodologías ágiles, la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa
- Posibilita el seguimiento y la coordinación del desarrollo, en lugar de controlar a los desarrolladores
- SCM responde a los cambios en lugar de evitarlos (manteniendo integridad y calidad)
- Cuando sea posible, debe automatizarse (ayuda a entregas frecuentes y continuas)
- Provee retroalimentación continua sobre calidad, estabilidad e integridad
- Las actividades de SCM están embebidas en las demás tareas para alcanzar el objetivo del sprint
- Requiere capacitación, ya que es responsabilidad de todo el equipo
- El manifiesto ágil lo que dice es como se va a trabajar en el contexto de proyectos de desarrollo y la gestión de configuración busca garantizar la calidad del producto (la gestión de configuración se debe hacer en todo proyecto, independientemente de si es ágil o tradicional, eso NO se discute, lo que se ve es como se va a implementar)
- Elimina el desperdicio, no agrega nada más que valor
- Provee documentación Lean y trazabilidad

INTEGRACIÓN, ENTREGA Y DESPLIEGUE CONTINUO

- Lo que se busca es automatizar lo más que se puedan las actividades del desarrollo
- **Integración continua**
 - o Cada desarrollador realiza pruebas unitarias (en la medida de lo posible automatizadas) esto lo hace mediante TDD (desarrollo conducido por pruebas) esto significa que cuando termino un componente de código lo prueba automáticamente para asegurarse que el software funciona y puede desplegarse en cualquier momento, y en ese momento lo sube al repositorio
 - o Esto ayuda a evitar problemas de incompatibilidad o de ejecución (ayuda a generar una versión del producto que está en condiciones de ir a las pruebas de aceptación sin mayores problemas)



- Entrega continua

- Se construye el software para que pueda liberarse a producción en cualquier momento (el software siempre esta en estado de entregable)
- Suma la automatización de las pruebas de aceptación, esto permite que el producto ya esté listo para ser desplegado a la producción en tiempos menores
- Se necesita como pre-requisito la entrega continua
- El producto puede pasar a producción más seguido , no se despliegan tantos cambios y disminuye el riesgo de que algo de rompa



- Despliegue continuo

- No existe una intervención humana para desplegar el producto en producción
- Se usan pipelines (serie de pasos a ejecutarse en un orden para una instalación satisfactoria)
- Esto permite que los cambios en el código se entreguen a los usuarios finales con mayor rapidez y frecuencia, reduciendo el tiempo de lanzamiento de nuevas funcionalidades y mejora la experiencia de los usuarios
- Algunas estrategias son el canary deployment o el blue/green deployment



SOFTWARE

- Información estructurada con propiedades lógicas y funcionales, creada y mantenida en varias formas y representaciones
- Esta confeccionada para ser procesada por computadora en su estado más desarrollado

El software evoluciona con el tiempo (pueden cambiarse el código, reglas, documentación, arquitectura, etc) este cambio puede implicar dificultades a la hora de mantener la integridad de un producto de software y es por eso que se usa la gestión de configuración

UNIDAD 3: FILOSOFIA O FRAMEWORK ÁGIL (NO DECIR METODOLOGIA)!

MANIFIESTO AGIL

- Es un documentado redactado en 2001 por expertos en el desarrollo de software, promueve la filosofía ágil en el desarrollo
- No es una guía de cómo hacer las cosas, es un compromiso que hacen las personas involucradas en un proyecto para trabajar de cierta manera, independientemente de las prácticas
- Lo que busca es lograr un equilibrio entre procesos rigurosos vs trabajar de forma eficiente y efectiva para entregar un producto de calidad al desarrollar software

FILOSOFIA ÁGIL

- Surge por el manifiesto ágil
- No es una metodología o proceso, es una ideología con un conjunto definido de principios que guían el desarrollo del producto (definen prácticas de cómo organizar un equipo)
- Tiene como objetivo entregar software funcional frecuentemente en un entorno cambiante
- Es un compromiso entre nada de proceso y demasiado proceso
 - o Se busca lograr un equilibrio entre seguir procesos rigurosos y formales (donde termina siendo más importante cumplir con el proceso que entregar software de calidad) y trabajar de manera eficiente y efectiva para entregar un producto de calidad
 - o No significa que no hay procesos, porque sin procesos todo seria un caos
- Se basa en procesos empíricos (usa como base a la experiencia que sale del equipo)
 - o Se usa en entornos con gran variabilidad de requerimientos, y se involucra al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación la cual nos permite realizar mejoras y correcciones continuas durante todo el proceso de desarrollo
 - o Usamos ciclos de realimentación cortos
- Usa ciclos de vida iterativos incrementales (serie de incrementos q añaden funcionalidad)
- Se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos
- Se diferencia del PUD en que si bien ambos son iterativo incremental, el PUD es definido
- Se orientan a la gente en lugar del proceso y son adaptables en lugar de predictivos
- Ejemplo de frameworks agile → FDD – ATDD – CRYSTAL – SCRUM

AGILISMO E INDISCIPLINADO

- Ser ágil no es ser indisciplinado, no significa trabajar sin orden ni sin planificación.
 - o **Mal agile** → no hay plan ni arquitectura, lo que genera desorganización y caos.
 - o **Enfoque tradicional** → se define todo desde el inicio con mucho detalle, lo que vuelve al proyecto rígido y poco adaptable a cambios.
 - o **Buen agile** → tener un plan y una arquitectura, pero de forma flexible y evolutiva, permitiendo ajustarlos a medida que el proyecto avanza. (no elimina la planificación, sino que la hace adaptable)

No es hacer todo por partes, lo que plantea es ir haciendo entregas tempranas y frecuentes de software pero que funcionen y devuelvan un valor de negocio que nos sirva independiente de si estamos lejos del producto final que queremos entregar

VALORES DE LOS EQUIPOS AGILES

- Planificación continua (multi nivel)
- Auto organizados y equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing recurrente

PORQUE IR A ÁGIL

- Acelera entregas de productos de valor para el cliente
- Permite adaptarse fácilmente a cambios de prioridades y requerimientos
- Permite alinear los objetivos organizacionales y de TI
- Permite mejor visibilidad del proyecto
- Reduce riesgos y costos de proyecto
- Mejora relaciones en el equipo

PILARES DEL EMPIRISMO

- **TRANSPARENCIA**
 - o Comunicación abierta y honesta de la información relevante a todas las partes interesadas
 - o Promueve la confianza y la colaboración entre los miembros del equipo y las partes interesadas, lo que conduce a una toma de decisiones más informada y eficaz
 - o Un artefacto con poca transparencia lleva a decisiones que disminuyen valor y aumentan riesgo
 - o Permite crecer como equipo y transformar el conocimiento implícito en explícito (el conocimiento pasa a formar parte del conocimiento de todo el equipo) deja de ser “mi código” y pasa a ser “nuestro código”
- **INSPECCIÓN**
 - o Revisión regular y sistemática del progreso del trabajo y los productos resultantes para detectar problemas y desviaciones del plan
 - o Permite una evaluación temprana y continua del progreso y ayuda a tomar decisiones informadas sobre los próximos pasos
 - o Inspeccionar sin transparencia es engañoso
- **ADAPTACIÓN**
 - o Es la mejora continua, la capacidad de adaptarse y ajustar el trabajo y el proceso para hacer frente a desviaciones y problemas detectados durante la inspección
 - o Permite una respuesta rápida y efectiva a los cambios y permite mantener la entrega de valor al cliente
 - o Deben hacerse los ajustes lo antes posible para minimizar la desviación normal
 - o Adaptar sin inspeccionar es inútil

AGILE ES APLICABLE EN EL CONTEXTO DE LO COMPLEJO, EN EL CUAL SE UBICAN LOS NUEVOS PRODUCTOS SOBRE LOS CUALES EL ENTENDIMIENTO Y LA CERTEZA QUE SE TIENE ES PARCIAL.

VALORES DEL MANIFIESTO AGIL

- Se priorizan ciertos aspectos sobre otros (pero sin restar importancia a ninguno de ellos)
- **Individuos e interacciones por sobre procesos y herramientas**
 - Damos más prioridad a los vínculos, comunicación efectiva y colaboración entre los miembros del equipo y las partes interesadas por sobre las herramientas a usar y el proceso a aplicar
 - No significa trabajar sin procesos ni herramientas, sino que buscamos procesos con altos niveles de interacción entre individuos por sobre los procesos y herramientas a utilizar
 - Es preferible un equipo de personas con buen funcionamiento, conocimiento, comunicación fluida y motivadas; pero con herramientas pobres para su trabajo, a que tener un equipo desmotivado o poco funcional pero con las mejores herramientas y procesos
 - Esto debido a que el desarrollo es una actividad humano intensiva
 - Los procesos ágiles buscan capitalizar las fortalezas de cada individuo, en lugar de homogeneizar un equipo; para lograr una armonía que fortalezca la unión y motivación
 - Los procesos y herramientas deben ser un complemento para mejorar la eficiencia y estos se deben adaptar al equipo y no al revés
- **Software funcionando por sobre documentación extensiva**
 - Es mejor priorizar el software para que cada iteración tenga software funcional y de calidad en lugar de enfocarse en documentar cada detalle del proceso
 - Hacer esto permite que cada iteración aporte valor al cliente y ayude con la retroalimentación
 - No quiere decir que no hay que documentar, existe la necesidad de mantener la información del producto y del proyecto que estamos desarrollando. Lo que se busca es que se genere la documentación cuando sea necesario y documentar aquello que agregue valor al producto
 - Las decisiones tomadas con respecto al producto de software deben documentarse y trascender más allá de la iteración en la cual se generó el incremento al mismo
 - Se debe decidir que aspectos van a documentarse y tener en cuenta la permanencia (no toda la info a generar debe tener permanencia y disponibilidad infinita)
 - Lo más importante es el acto de planificar, no el resultado
- **Colaboración con el cliente por sobre negociación contractual**
 - Es importante trabajar de manera colaborativa con el cliente, comprendiendo sus necesidades y requisitos en lugar de simplemente negociar contratos y acuerdos formales
 - Se relaciona con la triple restricción
 - En las negociaciones contractuales, cuando el cliente firma un contrato con el equipo definiendo alcances/costos/tiempos determinados y luego quiere realizar algún cambio en el mismo, surgen problemas, los cuales se pueden evitar si se involucra al cliente de manera más activa en el proyecto y se fomenta una actitud de colaboración e integración (el cliente debe estar dispuesto a participar, de lo contrario no se puede aplicar la metodología ágil)
- **Responder al cambio por sobre seguir un plan**
 - En lugar de tener una ERS firmada por el cliente, lo mejor es tener una actitud ajustada a la realidad (los requerimientos cambian, la gente se olvida, se da cuenta de lo que quiere cuando lo ve, surgen nuevas necesidades que no fueron detectadas en un principio, etc)
 - Los clientes que demandan software tienen un entorno cambiante, por eso un equipo ágil debe estar abierto a los cambios, para poder responder a ellos de forma eficiente y rápida,

entregando el mayor valor posible para el cliente y los usuarios (en lugar de apegarse a un plan rígido que fue definido en etapas tempranas de requerimientos y que puede no ser el mejor)

- Debe ser un equipo flexible y adaptativo, que no vea a los cambios como algo malo
- En un ambiente ágil se genera experiencia con cada iteración que puede afectar a la planificación de distintos niveles (este nuevo conocimiento puede incidir en nuevos cambios que el equipo ágil deberá incluir en la planificación, apuntando siempre a entregar producto con valor)

PRINCIPIOS DEL MANIFIESTO AGIL

- SATISFACER AL CLIENTE (3 y 4)

- La mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor
- El product owner va determinando la importancia y prioridad de cada funcionalidad para que cada release del producto tenga el mayor valor posible

- ACEPTAR QUE LOS REQUISITOS CAMBIEN (3 y 4)

- Los requerimientos cambian mucho, incluso en etapas tardías de desarrollo (debemos estar abiertos al cambio y ser flexibles, no seguir un plan estructurado)
- Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente

- ENTREGAS FUNCIONALES FRECUENTES (2)

- Entregamos software funcional con frecuencia, con preferencia a la documentación exhaustiva
- Es importante que, al finalizar cada iteración (que suelen ser periodos cortos de 1 a 4 semanas) el equipo haya sido capaz de transformar 1/+ requerimientos en código desarrollado, testeado y potencialmente desplegable (asegurarse que cumple con lo que el usuario necesita)

- TRABAJO EN EQUIPO (1 Y 3)

- Los responsables de negocios, diseñadores y desarrolladores deben trabajar juntos durante todo el proyecto para asegurarnos un software que satisfaga las necesidades del negocio
- Es importante el rol de Product owner y su trabajo en conjunto con el equipo de desarrolladores (este rol por lo general lo ocupa una persona del lado del cliente, que debe ser capaz de tomar decisiones sobre el producto, representar interés y necesidades comunes de todos los usuarios)

- PERSONAS MOTIVADAS (1)

- Los individuos que participan en el proyecto de desarrollo deben estar lo suficientemente motivados y comprometidos con los objetivos
- Los participantes deben verse entre sí como un único equipo con objetivos comunes, donde las personas que asumen diferentes roles intentan colaborar entre sí para cumplir con esos objetivos

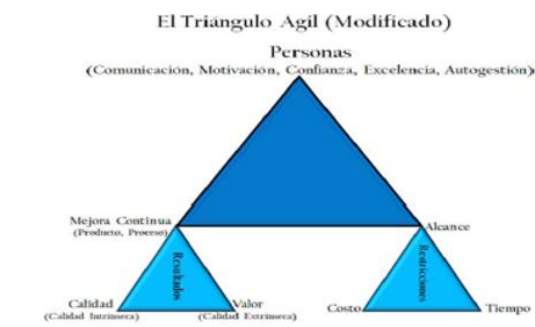
- COMUNICACIÓN DIRECTA (1)

- El método más eficiente de comunicar la información es conversación cara a cara (crece mucho la efectividad y la riqueza de la comunicación)

- Las palabras expresadas como sonido o texto representan un bajo porcentaje de la comunicación a lograr, la mayor parte está ligada a gestos faciales, movimientos corporales, etc
- **EL SOFTWARE FUNCIONANDO ES LA PRINCIPAL MEDIDA DE ÉXITO (2)**
 - La mejor forma de demostrar el avance en un proyecto de software es mediante la entrega de software funcional y de valor para el cliente (poco vale tener una excelente documentación si el producto no refleja el valor que se intenta entregar al cliente)
- **CONTINUIDAD (2 y 4)**
 - Los procesos ágiles promueven el desarrollo sostenible (el equipo debe ser capaz de mantener un ritmo constante y sostenible de trabajo a lo largo del tiempo)
 - Los framework ágiles apuntan a ciclos de vida iterativos de duración fija, si no llegué con todo lo previsto en la iteración entrego lo que tenga listo
 - La forma de llegar a que un equipo tenga un desarrollo sostenible, es lograr que el equipo tenga un ritmo de trabajo que asegure entregar en un ciclo de tiempo fijo
- **EXCELENCIA TÉCNICA (2)**
 - La atención continua a la excelencia técnica y al buen diseño mejora la agilidad
 - No es negociable la calidad del producto, debemos ajustar cualquier aspecto del producto menos la calidad a entregar
- **SIMPLICIDAD (2 Y 3)**
 - Al buscar simplicidad se puede maximizar la eficiencia y la efectividad, al mismo tiempo que se reduce la complejidad y el riesgo de errores y problemas
 - Se deben eliminar tareas innecesarias (no hay que perder el tiempo en aspectos que no se traducen en satisfacer requerimientos de usuario, no agrego funcionalidad si cliente no lo pidió) para concentrarnos en objetivos más importantes y críticos
- **EQUIPOS AUTO ORGANIZADOS (1)**
 - Las mejores arquitecturas, requisitos y diseños emergen de equipos auto organizados con libertad de tomar decisiones y diseñar soluciones de manera colaborativa
 - Es importante que los equipos de desarrollo tengan cierto grado de autonomía y capacidad de decisión en el proceso de diseño y desarrollo, en lugar de un enfoque rígido y jerárquico
 - Los más aptos para definir las características del producto son los que van a desarrollarlo (el conocimiento surge del equipo de trabajo)
- **Mejora continua (1 y 4)**
 - A intervalos de tiempo regulares, el equipo reflexiona sobre como ser más efectivo y de acuerdo a esto ajustan su comportamiento para adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo (para que el comportamiento del equipo como un todo mejore)
 - Se basa en la inspección y adaptación (permiten que los equipos evalúen distintos aspectos relacionados a su desempeño y reflexionar sobre oportunidades de mejora en general)

TRIANGULO AGIL VS TRIANGULO ÁGIL

- Triángulo de hierro o triple restricción (enfoque tradicional) **DIRIGIDO POR EL PLAN**
 - o Modelo utilizado en la gestión tradicional de proyectos, establece que los elementos fundamentales en cualquier proyecto son alcance, tiempo y costo (cualquier cambio en uno de estos elementos afecta a los otros dos)
 - o Se busca establecer un plan detallado en el cual, antes de empezar, se deja fijo el alcance y en base a eso derivó los recursos y costos para obtener un producto de calidad
 - o Primero se definen los requerimientos que formarán parte de cada iteración y a partir de esto, se determinan los recursos y el tiempo necesarios para llevarlos a cabo, por lo que la duración de cada iteración depende de la cantidad y complejidad de los requerimientos asignados.
- Triángulo ágil **DIRIGIDO POR VALOR**
 - o Se usa en metodologías ágiles
 - o Se considera que el alcance puede ir cambiando a lo largo del desarrollo de producto, por eso se dejan fijos el tiempo y los recursos y el alcance es variable
 - o Lo que hacemos es definir iteraciones de duración fija – sprints- (máximo un mes) y me pregunto ¿con los recursos y el tiempo que dispongo, que puedo entregarle al cliente en ese tiempo? Y en base a eso defino que features incluir en cada sprint (en base a la priorización que tienen y el valor de negocio que aporta cada una)
 - o Puedo adaptar al proyecto según necesidades y requisitos cambiantes
 - o Se busca maximizar el valor que se entrega al cliente, asegurar la entrega de software de calidad funcionando en los plazos establecidos (esto ayuda a bajar incertidumbre y lograr satisfacción del cliente ya que recibe software funcionando de forma frecuente)
- Triángulo ágil modificado
 - o Apunta a un producto de software que pone a las personas en lugar más importante (ya que las personas son lo más importante ya que hacen el software)
 - o Se prioriza comunicación, motivación, confianza, excelencia y autogestión
 - o Mejora continua (del producto y del proceso), es el triángulo de resultados
 - Características intrínsecas → características que las personas le ponen al producto para que pueda satisfacer al cliente
 - Calidad extrínseca → es el valor que el producto tiene para el cliente
 - o Alcance (restricciones de nuestro producto como el costo y tiempo que suelen ser grandes limitantes a la hora de desarrollar software)
 - o Finalmente se tienen en cuenta las variables que se usan para gestionar el trabajo (alcance-costo-tiempo)



UNIDAD 4: GESTIÓN ÁGIL DE REQUERIMIENTOS Y USER STORIES

REQUERIMIENTO ÁGILES

- Lo que más importa en agile es construir el producto correcto, que le de valor a mi cliente
- En lugar de definir todos los requerimientos al principio (enfoque tradicional), los vamos descubriendo a medida que avanza el proyecto (definimos lo mínimo necesario para poder empezar a trabajar, y vamos avanzando en la definición mediante retroalimentaciones q ayudan a mejorar)
- Es importante trabajar en conjunto con el cliente en este proceso de descubrimiento
- El cliente va expresando los requerimientos necesarios mediante historias de usuario
 - o Son breves descripciones de los requisitos del cliente que se escriben en lenguaje natural y se enfocan en las necesidades del usuario
 - o Se usan para definir el alcance de cada iteración y se priorizan según el valor que aportan al producto (se debe plantear que valor tiene para el cliente cumplir cada requerimiento)

BIG REQUIREMENTS UP FRONT BRUF

- Todos los productos de software tienen un desperdicio significativo (hasta los más exitosos) esto significa que de todas las funcionalidades desarrolladas, un alto porcentaje de ellas no son usadas por el cliente/usuario (perdida de esfuerzo)
- Es por eso que en una primera instancia de un proyecto de software, con desarrollar unas pocas funcionalidades de gran valor e importancia para el cliente, se puede cubrir gran parte de sus necesidades
- No hay que hacer cosas “ya que estamos” si no nos las pidieron



PRODUCT BACKLOG

- La gestión ágil busca evitar el problema de desarrollar software que luego no se utiliza, enfocándose en construir únicamente aquellas funcionalidades que realmente aportan valor al cliente
- Para lograrlo, los requerimientos se organizan en una lista priorizada (Product Backlog), que es gestionada por el Product Owner, quien tiene la responsabilidad de entender las necesidades del negocio y decidir qué se debe hacer primero según el valor que los req. tengan para el negocio
- Los requerimientos se ordenan según su valor: los más importantes se ubican en la parte superior y se trabajan con mayor nivel de detalle, mientras que los de menor prioridad quedan en la parte inferior. A medida que el equipo avanza, va tomando los elementos de mayor prioridad y, una vez completados, continúa con los siguientes en orden
- Durante el desarrollo, pueden incorporarse nuevos requerimientos, eliminarse otros o modificarse sus prioridades según cambien las necesidades del negocio. De esta manera, los elementos de menor prioridad permanecen flexibles y sujetos a cambios hasta que sea necesario abordarlos
- En el enfoque tradicional el usuario no se compromete en priorizar y lo deja a libre decisión de los desarrolladores (esto trae problemas ya q después viene y se queja de xq no se priorizó x cosa)

MINI RESUMEN DE GESTIÓN DE REQUERIMIENTOS

En otras palabras, la gestión ágil de requerimientos implica trabajar junto a técnicos y no técnicos entendiendo las necesidades y el negocio para construir un software que de valor. Y luego, junto con los usuarios descubrir cuál es la mejor forma de satisfacer esas necesidades (Aquí aparece el Product Backlog). Entregada la característica, obtenemos feedback que nos va a ayudar a mejorar el Product Backlog si es necesario.

JUST IN TIME

- El producto no está especificado al 100% al principio ni mucho menos, con el tiempo se van especificando los requerimientos cuando sea necesario junto a la ayuda del cliente que me brinda retroalimentación
- Hay que analizar los requerimientos cuando sea necesario, cuando tenga suficiente valor (no puede ser ni antes ni después), es importante tomar la decisión en el momento que haya que tomarla para evitar desperdicios (ejemplo de desperdiciar sería especificar un requerimiento que después cambia)
- Hay que entregarle al cliente valor de negocio, no características de software, el software es el medio por el cual le entregamos valor de negocio

FUENTE DE REQUERIMIENTOS

- La mejor forma de obtener requerimientos de software es mediante la conversación cara a cara con el cliente ya que permite analizar mejor los gestos que manifiesta el cliente y es un medio de comunicación mucho más enriquecedor (la riqueza de la comunicación va creciendo exponencialmente al pasar de papel a audio, audio a video, video a llamada, llamada a cara cara)
- En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo mientras que en agile no se hace foco en documentar (visto antes)

TIPOS DE REQUERIMIENTOS

- A nivel de la gestión ágil de requerimientos se trabaja con requerimientos de negocio y de usuario
- Al usuario le interesa el requerimiento de negocio (el valor de negocio es el requerimiento de negocio, pero el medio por el cual logramos ese valor de negocio es el requerimiento de usuario)
- Las US sirven para identificar requerimientos de usuario (no para especificar requer. de software)
- **Tipos de requerimientos**
 - **De negocio**
 - Son los de más alto nivel, son los requerimientos en términos de la visión del negocio
 - Ejemplo → disminuir un x% de tiempo invertido en procesos manuales
 - **De usuario**
 - Tienen que ver con los requerimientos de usuario final
 - Ejemplo → realizar consultas en línea
 - **Funcionales**
 - Se pueden relacionar 1 a 1 con los casos de uso de la gestión tradicional
 - Ejemplo → generar reportes
 - **No funcionales**
 - Suelen ser los más ocultos, que el cliente no tiene en claro y debo especificarlos
 - Un RNF puede hacer que el software que estoy desarrollando no sirva (importancia fundamental)
 - Ejemplo → formato de un reporte
 - **De implementación**
 - Se suele decir que están dentro de los no funcionales, pero tienen que ver con cuestiones de restricción o implementación específicos
 - Ejemplo → servidores en la nube

NO SE EL TITULO



- **Dominio del problema → requerimientos de negocio y de usuario**
 - o En esta capa se centra el manifiesto agile
 - o No decimos que debería hacer el sistema, sino que ponemos foco en que cosas quiere hacer nuestro usuario y cual es el valor de negocio que espera obtener como producto de eso
 - o Es lo que al cliente le interesa
- **Dominio de la solución → requerimientos de software**
 - o No se habla de que quiere el usuario, ni que espera; sino se habla en términos de software (es como el equipo transforma las necesidades del cliente en código ejecutable)

Es importante identificar los requerimientos que tienen que ver con el dominio del problema y luego de la solución (esta distinción es muy importante ya que es fundamental para entender las necesidades del usuario y las restricciones del contexto en el que se usa el software, permitiendo a los equipo de desarrollo adaptarse rápidamente a los cambios y priorizar características más importantes del software)

QUE SE DEBE ASUMIR

- **Los cambios van a existir todo el tiempo de forma constante**
 - o Siempre hay cambio y debemos gestionar los requerimientos orientados a poder aceptar el campo
 - o Si no se plantean cambios significa que el producto no se está utilizando
- **Debemos tener una mirada de todos los que están involucrados**
 - o Son todos los que tienen algo para decir del producto (stake holders)
 - o Para que esto no sea caótico, el PO es el representante de todos los que tienen algo que decir sobre el producto (solo existe un PO) y es el que logra los acuerdos con los stake-holders
- **El usuario dice lo que quiere cuando recibe lo que pidió**
 - o Por este motivo tener iteraciones cortas y entregas continuas que permiten retroalimentación y enriquecimiento para futuras entregas es algo esencial
- **No hay técnicas ni herramientas que sirvan para todos los casos**
 - o Debemos detectar la herramienta más adecuada para la situación específica
- **Lo importante no es entregar una salida, un requerimiento, lo importante es entregar un resultado, una solución de valor**
 - o Siempre se debe entregar soluciones que den valor al cliente, minimizando el desperdicio

PRINCIPIOS ÁGILES RELACIONADOS A LOS REQUERIMIENTOS ÁGILES

- La prioridad es satisfacer al cliente mediante releases tempranos y frecuentes (2 semanas a un mes)
- Recibir cambios de requerimientos, aun en etapas finales
- Técnicos y no técnicos trabajando juntos todo el proyecto (el PO es parte del equipo, porque si los no técnicos no son parte del equipo del proyecto, difícilmente pueda identificar que da valor al negocio)
- El medio de comunicación por excelencia es cara a cara
- Las mejores arquitecturas, diseños y requerimientos emergen de equipos auto organizados (le damos responsabilidad al equipo y que tome las decisiones técnicas)

USER STORIES

- Descripción corta de una necesidad que tiene el usuario respecto a un producto de software
- Es una técnica para capturar requerimientos en ambientes ágiles
- Surgen para disminuir la dificultad que existe al momento de comunicar lo que hará el sistema a todas las personas afectadas.
- Las escribe el PO y las prioriza en el backlog
- La parte más difícil de construir un sistema de software es decidir que construir
 - o Ninguna otra parte del trabajo conceptual es tan difícil como establecer los requerimientos técnicos detallados (debido a incertidumbre, cambio constante, mala comunicación)
 - o Ninguna otra parte del trabajo afecta tanto el sistema resultante si se hace incorrectamente
 - o Ninguna otra parte es tan difícil de rectificar más adelante
- Decimos que son multipropósito ya que cumplen varias funciones en el desarrollo de un software, las US son:
 - o Una necesidad de usuario (especifican que necesita el usuario)
 - o Una descripción del producto (describe las características especificadas por el usuario)
 - o Un ítem de planificación (según su prioridad se da el orden de implementación)
 - o Un token para una conversación (cuando llega al tope de la pila, tenemos que sentarnos a especificar los detalles de esa user)
 - o Un mecanismo para diferir una conversación (sirve de recordatorios para conversaciones futuras sobre características a construir del producto)
- No son especificaciones detalladas de requerimientos, ni reemplazan a la ERS únicamente expresan la intención de hacer algo (no necesitan descripciones exhaustivas de la funcionalidad del sistema)
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación
- Junto al código, sirven de entrada a la documentación que se desarrolla incrementalmente después
- Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual
- Funciona como un recordatorio para el equipo de desarrollo y el PO de que tienen que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor

PARTES DE UNA USER (LLAMADAS LAS 3 C EN INGLES)

- **Conversación**
 - o Es la parte más importante de la user
 - o Es el momento en el cual el equipo de desarrollo dialoga con el PO para capturar las necesidades del problema y poder definir que funcionalidades vamos a agregar a nuestro sistema (captamos todos los detalles necesarios)
 - o Asociada con acuerdo sobre la negociación y técnicos y no técnicos trabajando juntos
 - o Por lo general no queda escrita en ningún lado (decimos que es la parte no visible) aunque puede grabarse, guardarse en minutas, etc

- Confirmación

- Es un acuerdo entre el PO y el equipo de desarrollo para demostrar que la característica realmente se implementó
- Conformar la definición de pruebas de aceptación (pruebas que define el PO) las cuales una user debe pasar para considerarse terminada

- Tarjeta

- Es la parte visible de la user, es donde escribimos o expresamos algo que nos indique cual es la historia de usuario, de que trata y cuál es su valor de negocio
- Su sintaxis es la siguiente

<<Frase Verbal>>
Como <<nombre del rol>>  puedo <<actividad>> de forma tal que <<valor de negocio>>

- **Frase verbal**
 - Nos ayuda a identificar de que trata la user sin tener q leerla toda
- **Nombre del rol**
 - Quien realiza la acción (quien recibe el valor de la actividad)
 - Evitar poner usuario (es algo muy genérico)
- **Actividad**
 - Acción que realizará el sistema
- **Valor de negocio**
 - Es la parte más importante, representa el porqué es necesaria la actividad
 - Nos permite priorizar users
- Tiene también criterios y pruebas de aceptación pero eso lo defino más adelante

PRODUCT BACKLOG Y LAS USER STORIES

- Es una pila de product backlog ítems con sus correspondientes estimaciones de tamaño
- El PO prioriza las historias en el Producto Backlog (lista de todos los requerimientos que tiene que cumplir mi producto y con los que vamos a ir trabajando)
- Las US que están
 - En el tope de pila → tienen mayor prioridad, y están detalladas con una granularidad fina
 - En el fondo de la pila → tienen menor prioridad, y están detalladas con una granularidad gruesa (no se tiene mucho detalle ya que no son indispensables)
- En cualquier momento se puede cambiar esa priorización, según como lo indique el PO (que es el encargado del proceso de priorización) también pueden agregarse, quitarse, modificarse
- El PO nunca contiene el 100% de los requerimientos de usuario

LAS USER SE IMPLEMENTAN COMO PORCIONES VERTICALES.



- Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos (debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada)
- Si fuesen horizontales no entregarían valor (solo entrego base de datos o gui o lógica de negocio y eso no tiene valor para el usuario)

MODELADO DE ROLES

- Es importante conocer y entender quién va a ser el usuario, para poder diseñar la experiencia de usuario pensando en ese perfil específico (El rol nos ayuda a identificar user story, que necesidades de negocio tiene ese rol y que cosas va a querer hacer)
- Intentamos modelar los roles dentro del sistema para lograr identificar aquellos que probablemente aparezcan en las user story
- Un rol define la actividad a ejecutar y el valor de negocio (pueden existir actividades que parecen similares pero en función de su rol sus valores son diferentes y sus US son diferentes)
- Usamos las tarjetas como rol de usuario, ahí anotamos las características en general que va a tener el usuario (hay que evitar usar como rol de usuario el nombre usuario)
- **Una técnica adicional es la de personas**
 - o Lo que hago es pensar en una descripción de una persona concreta que va a usar el sistema
 - o Si bien en sistemas grandes (como la facu) esto es complejo, buscamos modelos genéricos (el que hace la carrera al día, el que debe solo finales, el que abandonó y volvió., etc) y así adquirimos info

PROXIES (USUARIOS REPRESENTANTES)

- Cuando el product owner no está disponible, tenemos que buscar alternativas a el
- Esas personas deben ser perfiles de negocio (NO INFORMÁTICOS) y deben poder tomar decisiones
- Esta es una lista de personas que no son ideales como representantes, evitarlos!
 - o Gerente de usuarios o desarrollo
 - o Alguien de marketing
 - o Vendedores
 - o Expertos en el dominio
 - o Clientes
 - o Capacitadores y personal de soporte

CRITERIOS DE ACEPTACIÓN

- El product owner define los criterios que debe cumplir el software para aceptar la implementación
Debe escribirse en términos de objetivos concretos
- Se definen los requerimientos funcionales mínimos (puede ser una cantidad ilimitada) para que la user provea valor al negocio
- Debe tener una redacción clara, tienen que haber validaciones concretas
- Se usan las palabras puede (opcional) y debe (obligatorio)
- Se escribe a nivel usuario
- **Ayudan a varias cosas**
 - o El equipo puede tener una visión compartida de la user
 - o Define límites para una user
 - o Permite saber si lo que implementamos es o no lo correcto
 - o Ayudan a desarrolladores y testers a derivar pruebas
 - o Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad
- **Son buenos cuando**
 - o Son independientes de la implementación
 - o Son relativamente de alto nivel, no es necesario que se escriba cada detalle
 - o Definen una intención, no una solución

- ¿Dónde van los detalles?
 - o Los detalles son el resultado de las conversaciones con el Po y el equipo puede guardarlo en:
 - Documentación interna de los equipos
 - Pruebas de aceptación automatizadas
 - o Cada equipo decide donde y como guardar el detalle
 - o Ejemplos de detalle → el formato es dd/mm, debe usarse una lista y no un check box, etc

PRUEBAS DE ACEPTACIÓN

- Se encuentran al dorso de la tarjeta y complementan la historia
- Expresan detalles de la conversación entre clientes y desarrolladores
- El PO acepta la user como implementada si todas las pruebas de aceptación pasan
- Profundiza lo definido en la user y ayuda a terminar de definir como implementarla
- Se agrega lo que esperamos que pase cuando se ejecute la historia de usuario (entre paréntesis se pone pasa o falla refiriéndonos al resultado del test, se prueban escenarios exitosos y fracasos)
- Se encuentran las más importantes, ya que no es posible escribir todas las posibles
- Deben escribirse antes que la programación empiece (y se recomienda q las escriba el cliente)

TAMAÑO

- Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños antes que tener historias muy grandes
- El tamaño de las users se mide en story points
- Aspectos que determinan el tamaño de una user
 - o Complejidad
 - Hace referencia a una dificultad inherente a la funcionalidad que no permita dividir las en users más pequeñas
 - o Esfuerzo
 - Tiempo en horas de trabajo que requerirá la implementación
 - o Incertidumbre
 - Técnica → dudas en el dominio de la solución (usar x tecnología no conocida por el equipo)
 - De negocio → no se conoce como interactuará el usuario con el sistema, causado por falta de conocimiento del dominio del negocio

DOR (DEFINITION OF READY) – DEFINICIÓN DE LISTO – CRITERIO DE READY

- Es un criterio que permite definir que tiene que cumplir una user para asegurarnos que esta lista para ser implementada (lista para entrar en una iteración de desarrollo o sprint)
- Para que la user pueda implementarse mínimamente debe satisfacer el INVEST model, aunque el equipo de desarrollo y el PO puede definir características extras para que una user se considere lista
- Si no cumple con eso debo trabajar mas en esa user

INVEST MODEL

- Modelo de calidad que nos ayuda a verificar si la user cuenta con las características de calidad mínima para satisfacer la DOR y poder ingresar en una iteración
- **Independiente**
 - o Una user debe poder implementarse en cualquier orden y sin que dependa de otra user
 - o El PO tiene libertad de priorizarlas en el product backlog sin afectar otra user
- **Negociable**
 - o Se debe escribir una user en función de QUE necesita el usuario no el COMO la vamos a implementar
 - o Por eso en los criterios de aceptación no se ponen detalles de implementación
- **Valuable**
 - o Debe tener un valor concreto para el cliente (es el para que de la descripción)
 - o Deben aportar valor al negocio a las personas a las que está destinado el producto
- **Estimable**
 - o Se debe poder estimar el tamaño, complejidad y esfuerzo que llevara hacer la user (si no puedo estimar cuanto tiempo dura no puedo determinar si puede entrar en una iteración y por lo tanto no puedo construirla)
 - o Puede no ser estimable porque
 - El equipo tiene incertidumbre del dominio (debe discutir con el PO para sacar dudas)
 - El equipo tiene incertidumbre técnica, ósea del dominio de la solución (asignamos spikes)
 - La user es muy grande (debemos dividirla)
- **Small**
 - o La user debe ser lo suficientemente pequeña para ser finalizada en una iteración (sprint)
 - o El tamaño es relativo, depende de la experiencia y capacidad de equipo y de la duración de la iteración
 - o No se nos debe ir la mano con lo pequeño para que entre en una iteración porque si es muy pequeña no genera valor al negocio y eso no sirve
- **Testeable**
 - o Se debe poder demostrar que la user fue implementada cumpliendo los criterios de aceptación

DEFINICIÓN DE HECHO

- Determina si una user esta terminada, son los criterios que establecen cuando una user story ha sido implementada de forma correcta y completa para mostrársela al PO para que decida si pasa o no a producción
- Cuando esa user cumple con la definición de hecho consideramos que está lista para desplegarse
- En la práctica es un checklist con las condiciones a cumplir y es propia del equipo
- Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una Definición de Hecho adecuada para el producto.

NIVELES DE ABSTRACCIÓN

- **User story**
 - o Cumple con el criterio de listo (DOR) para ingresar a una iteración y están en el backlog
- **Épica**
 - o Es una user muy grande que no puede entrar en una iteración (usualment no cumple INVEST)
 - o Se deben dividir en user más pequeñas llegado el momento de su implementación
 - o Sirve para los requerimientos que no son prioridades en las primeras iteraciones y basta con tener solo una aproximación de las funciones generales
 - o Permiten reducir el esfuerzo del equipo al estimar
- **Temas**
 - o Conjunto de users agrupadas, ya que están relacionadas
 - o Simplemente le ponemos un nombre para saber que es algo sobre lo que tenemos que hablar en algún momento (recién cuando suba en la pila me pongo a detallarlo)
 - o Permiten reducir el esfuerzo del equipo al estimar



SPIKES

- Es un tipo especial de user story, sirven para quitar incertidumbre sobre otro requerimiento de otra user story o alguna faceta del proyecto a investigar (se crean específicamente para esto)
- Si existe incertidumbre → no puede estimarse → no cumple con DoR
- Una vez resuelta la incertidumbre, la Spike se convierte en una o más US
- Ejemplos: familiarizarse con una nueva tecnología – analizar comportamiento de una user compleja – ganar confianza frente riesgos tecnológicos – enfrentar riesgos funcionales
- **Se clasifican en**
 - o Técnicas
 - Usadas para investigar enfoques técnicos sobre el dominio de la solución (evaluar performance, decisión de hacer o comprar, evaluar implementar x tecnología)
 - Situaciones en las que el equipo necesita una comprensión más fiable de alguna tecnología a aplicar antes de comprometerse a desarrollar una nueva funcionalidad en un tiempo fijo
 - o Funcionales
 - Usadas para cuando hay incertidumbre de cómo el usuario interactuará con el sistema
 - Son mejores evaluadas mediante prototipos para obtener retroalimentación
- **Lineamientos**
 - o Deben ser estimables, demostrables, aceptables y cumplir con el criterio de ready
 - o Es una excepción, no la regla (no es que siempre que haya incertidumbre hago una spike, lo voy a usar como última opción, cuando realmente hayan incógnitas muy grande)
 - o Se debe trabajar la spike en una iteración separada de las historias resultantes
 - En general se trabaja la spike en un sprint anterior al cual yo quiero implementar la user
 - Salvo que el spike sea pequeño, sencillo y sea probable encontrar una solución rápida
- **Algunas user pueden requerir de ambos tipos de spikes (por ejemplo quiero ver el uso diario en un histograma de mi consumo de energía, esto requiere un spike)**
 - o Spike técnico → investigar cuanto tiempo requiere actualizar un display de un cliente al uso actual
 - o Spike funcional → crear un prototipo de histograma en el portal web y obtener retroalimentación del usuario sobre tamaño, estilo y atributos

ALGUNAS COSAS A TENER EN CUENTA

- Se debe diferir el análisis detallado tan tarde como sea posible (justo antes que el trabajo comience) esto significa que lo veo cuando sea necesario (ni antes ni después) para evitar desperdicios
- Las user story se hacen de un paso a la vez (evitar la palabra “y”) se deben usar palabras claras en los criterios de aceptación
- Se deben escribir las user desde la perspectiva del usuario
- No se debe forzar todo para escribirlo como user (no todo va en la user)

FRACASO EN AGILE

- En agile la participación del PO es de suma importancia, ya que es quien guía y prioriza en cada iteración los requerimientos que serán implementados
- Además por lo general es el encargado de escribir las user (que sirven de recordatorio y guía para el equipo de desarrollo para saber que deben implementar)
- Comúnmente el PO no posee esta disponibilidad que plantea agile, convirtiéndose en el talón de Aquiles en el proceso de desarrollo de software y siendo uno de los principales motivos por los que se fracasa en enfoques ágiles

UNIDAD 5: ESTIMACIONES TRADICIONALES

QUE ES ESTIMAR

- Al planificar en un proyecto de software se deben hacer estimaciones
- Es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto
- Nos ayuda a saber cuanto durará o costará un proyecto y se considera una de las act. más complejas
- Estimar NO es planificar (estimar sirve para planificar pero el plan no tiene por qué coincidir con lo estimado) y NO es un compromiso
- Es predecir en un momento en el que tenemos poco conocimiento y hay incertidumbre (por eso decimos que no es precisa y que depende de la cantidad de info que dispongamos, mientras avanzamos en el proyecto nuestra incertidumbre va disminuyendo - ahí deberíamos corregir -)
- Mientras más diferencia exista entre lo estimado y lo planeado, hay más riesgo
- Para estimar nos sirven datos históricos (comparar nuestro software a desarrollar contra softwares ya desarrollados con características similares), personas con experiencia y el valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe

PARA QUE ESTIMAMOS

- Existen dos razones por las cuales estimamos: predecir completitud y administrar riesgos
- La mayoría de las veces el proyecto termina en orden de 4 veces más de lo que uno pensó (en términos de tamaño, esfuerzo, tiempo y plata) muy pocas veces terminan o cuestan menos
- Casi siempre nos obligan a dar estimaciones tempranas (cuando no tengo suficiente información para estimar) es por eso que debemos postergar la situación de estimar cuanto más podamos para así tener la información necesaria que me disminuya el grado de incertidumbre
- Las estimaciones requieren experiencia, acceso a buena info histórica
- Antes de que el proyecto comience, el equipo de software debe estimar lo siguiente (en orden)
 - o **Tamaño del software**
 - Responde el que
 - Definimos que tan grande va a ser el trabajo que vamos a hacer
 - Para eso usamos la técnica de contar (funcionalidades, requerimientos, user, CU)
 - Es lo primero que se estima ya que afecta a todo el resto
 - La mayoría de los errores recaen en estimar los de abajo sin un tamaño previo
 - o **Esfuerzo**
 - Responde como
 - Se mide en cantidad de horas lineales (sin multitasking) e ideales (no tengo en cuenta tiempo de almuerzo, óseo, etc)
 - Lo afecta el tamaño (mas grande necesito mas gente)
 - o **Tiempo calendario**
 - Responde el cuando
 - Es calendarizar el esfuerzo y proponer una fecha de entrega (no es estimar a nivel de día, sino una aproximación necesaria) para que el cliente tenga una idea de la demora
 - o **Costo**
 - Depende del tamaño y del esfuerzo que se necesite, por eso es lo último
 - Ayuda a dar idea de cuál es el costo aproximado del proyecto

ERRORES COMUNES AL ESTIMAR

- Siempre existe un grado de error al estimar, ya que vivimos en un universo de incertidumbre
- Cuando hay incertidumbre existe error, y si hay error existe la chance de errar en algo (de todas maneras es mejor estimar que no estimar, incluso aunque estemos con riesgos de errar)
- Errores en las estimaciones
 - o No tenemos en claro el proceso a usar para trabajar
 - o Falta de información (asumimos ciertas cosas que en realidad no sabemos)
 - o El proceso de estimar tiene sesgo, si bien siempre buscamos las técnicas con menos margen de error, dicho margen existe
 - o Actividades omitidas (por ejemplo estimamos solo el tiempo de programación pero olvidamos el Testing, reuniones con el cliente, etc)
 - o No tenemos en claro los recursos que la empresa cuenta para desarrollar el producto

TECNICAS DE ESTIMACIÓN

- Existen distintas formas de estimar que nos ayudan a disminuir la incertidumbre (no la eliminan pero la reducen considerablemente)
- Cada técnica tiene ventajas y desventajas, es por eso que se la debe elegir según el proyecto específico
- Tipos
 - o **Métodos basados exclusivamente en recursos**
 - Se basa en la cantidad y tipo de recursos (personas, tiempo, herramientas, etc) necesarios para llevar a cabo el proyecto
 - Se estima en función de la disponibilidad y costos de los recursos necesarios
 - o **Métodos basados exclusivamente en el mercado**
 - Se pone el foco en el cliente y se realizan estimaciones en función de su capacidad de pago (en definitiva no estamos estimando)
 - No se evalúan el producto que se intenta vender, sino a quien se lo vende
 - A veces se cobra de menos para que nos elijan antes q a la competencia
 - Es diferente lo que vale mi producto, lo que cuesta y lo que el cliente puede/quiere pagar
 - o **Métodos basados en los componentes del producto o en el proceso de desarrollo**
 - Se basa en identificar y descomponer los componentes del software y del proceso de desarrollo para estimar esfuerzo y tiempo requerido para cada componente
 - Luego sumo las estimaciones y obtengo una estimación total
 - o **Métodos algorítmicos**
 - Se ejecutan algoritmos que tienen ciertos parámetros de entrada (tamaño, complejidad, riesgo, expertiz de la gente) y en base a eso calcula y da un valor
 - Siempre existen factores multiplicativos asociados a cada variable, es por eso que el resultado (a pesar de ser una fórmula matemática) muchas veces esta “forzado” para que de lo que tiene que dar

- **Métodos basados en la experiencia**

- Se basa en la experiencia pasada del equipo en proyectos similares para determinar el esfuerzo y tiempo requerido para un nuevo proyecto, tiene varios tipos
- Datos históricos
 - Consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones
 - Se busca una alternativa en la que el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona
 - Se contradice con agile (la experiencia no puede extrapolarse a otro equipo)
 - Tomo de entrada tiempo, tecnología usada, perfil de las personas y esfuerzos
- Juicio experto
 - Se basa en la experiencia y conocimiento de expertos en la materia para estimar el esfuerzo y tiempo requerido para un proyecto de software (es subjetivo y puede variar según el nivel de experiencia)
 - Es uno de los más utilizados
 - Se puede aplicar mediante dos técnicas:
 - Experto puro
 - Un experto estudia las especificaciones y realiza una estimación
 - La estimación se basa en la experiencia del experto (el problema es que si el experto se va de la organización, esta pierde su capacidad de estimación)
 - Es el mas usado en el 75% de organizaciones
 - Pero... experto en qué? (en un producto de software existen varias cosas en las que ser expertos-codigo, dominio, metodologías, etc-)
 - Se usa el metodo optimista, pesimista, habilidad (se le pregunta al experto sobre esos 3 valores y se aplica una fórmula para estimar que se calcula como $(O + 4H + P) / 6$)
 - Wideband delphi
 - Es similar al anterior pero se hace de manera grupal (un grupo de personas se reúnen para convergir a una estimación común)
 - Es mejor estimar en grupo que individualmente
 - Un grupo de expertos analiza el problema y genera estimaciones individuales en secreto, luego se ponen en común las diferencias numéricas extremas se discuten las diferencias y se re-estiman (se repite hasta que todos estén de acuerdo, y si en más de 2 vueltas no converge el líder hace un promedio o ve cuál se repitió más)
 - Lo malo es que lleva más tiempo y recursos que el experto puro
- **Analogía**
 - También se basa en datos históricos, pero se seleccionan específicamente aquellos similares al proyecto actual para realizar la comparación y estimación

ACTIVIDADES OMITIDAS

- Una de las fuentes de error más común en las estimaciones es omitir actividades necesarias para estimar el proyecto, por ejemplo:
 - o Requerimientos faltantes
 - o Actividades de desarrollo faltantes (documentación técnica, participación en revisiones, creación de datos para el Testing, mantenimiento de producto en previas versiones)
 - o Actividades generales (días de enfermedad, licencias, cursos, reuniones de compañía)
- Al estimar los desarrolladores suelen ser mas bien optimistas, siempre generan cronogramas optimistas (eso tiene que ver con que las estimaciones de software son algo nuevo y muchas veces subestimamos las dificultades que nos podemos encontrar)

UNIDAD 5: ESTIMACIONES AGILES

ESTIMACIONES AGILES

- Quien estima es quien debe hacer el trabajo, por ese motivo el equipo de desarrollo es el que estima de forma colectiva (sin incluir al PO). De esta manera podemos detectar otros puntos de vista que nos permitan estimar de la mejor forma posible (a diferencia de agile que se estimaba individual)
- Para esto se debe tener una mente abierta para escuchar a los demás y no cerrarse en nada
- Debemos recordar que no debemos ser dogmáticos sobre nada (seguir reglas de forma rígida) y que la clave del éxito en casi todo, en contraposición es ser pragmáticos (basarse en la experiencia)
- La estimación no es un plan, ya que un plan implica compromiso y si la estimación implicar un compromiso va a resultar perjudicial para cualquier organización.
- Es la parte más difícil luego de la elicitación
- Tips sobre las estimaciones que cambian respecto a lo tradicional
 - o Si las estimaciones se usan como compromisos son muy peligrosas y perjudiciales
 - o La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no
 - o Lo más beneficioso es el proceso de hacerlas
 - Lo más importante es el proceso de hacer, al estimar se enriquece el trabajo del equipo y mejora futuros procesos de estimaciones
 - o La estimación puede servir como gran protección para el equipo
 - Si decimos que tenemos 3 días para desarrollar el producto entonces no podemos decir que vamos a tener 160 hs de trabajo
- Frente a la posibilidad de estimar mal y generar enojo por parte del cliente, podemos
 - o Simplificamos, sin considerar todo el trabajo que hay que hacer, por miedo a que si el valor es más alto el cliente lo rechace.
 - o Sobreestimar, cuando me preguntan por ejemplo cuantas horas me va a llevar un trabajo le agrego más tiempo del que yo pienso

En ágil, no se busca dedicar mucho esfuerzo a la estimación porque más esfuerzo no garantiza mayor precisión. La relación entre esfuerzo y exactitud no es lineal: con poco esfuerzo ya se obtiene una buena aproximación. Por eso, se define un tiempo limitado para estimar y evitar desperdicio (la idea clave es lograr un equilibrio: invertir lo justo para obtener estimaciones suficientemente confiables sin gastar más recursos de los necesarios.)

COSAS A TENER EN CUENTA

- Vamos a trabajar estimando las stories mediante una medida de tamaño relativa llamada story points (esto separa la estimación de esfuerzo de la duración del proyecto)
- Los humanos no sabemos estimar de forma absoluta ya que se producen muchos sesgos o cuestiones relacionadas con quien estima pero somos muy buenos trabajando con estimaciones relativas (surgen de una comparación)
- Trabajar de forma relativa además es más rápido
- Es mejor estimar en grupo (obtenemos mejor dinámica grupal y pensamiento)

QUE SE ESTIMA

- Estimamos el tamaño de la user usando como unidad de medida los story points y teniendo en cuenta los 3 aspectos de esta (complejidad, esfuerzo, incertidumbre o duda)
- **¿Por qué no estimamos en tiempo?**
 - o Las estimaciones basadas en tiempo son más propensas a errores debido a varias razones:
 - Habilidades y conocimientos
 - Experiencia
 - Familiaridad con los dominios de aplicación / negocio
 - o Un trabajo puede requerir un tiempo de x horas pero transcurridas esas horas no significa que esté terminado (en medio pueden pasar ciertas cosas)
- **Tamaño**
 - o Es una medida de la cantidad de trabajo que se necesita para producir o terminar una user, indica
 - Cuan compleja es una user
 - Cuanto trabajo se requiere para completarla
 - Cuan grande es
 - o No es cantidad de código
 - o Magnitud general + esfuerzo físico o mental + complejidad arquitectónica
 - o Es independiente de quien lo haga, depende de qué implica
 - o Formas de estimar tamaño
 - Números del 1 al 10 / talles de remeras / serie exponencial de base 2 / serie de Fibonacci
 - Una vez definida la escala NO SE CAMBIA, si la cambiamos debemos cambiar el patrón de medición
- **Complejidad**
 - o Que tan compleja es implementar esa user story
 - o Incluye esfuerzo físico y mental y la complejidad arquitectónica o del código a construir
- **Esfuerzo**
 - o Es importante estimar las personas que van a hacer el trabajo ya que el esfuerzo es distinto entre dos personas
 - o Son las horas persona lineales e ideales que se requiere para terminar la user
 - o Es el factor más costoso de un proyecto
- **Incertidumbre o duda**
 - o Es cuanta información falta para ser mas adecuado en la estimación
 - o Nivel de incertidumbre, riesgo o incógnita

NO CONFUNDIR

- No hay que confundir tamaño con esfuerzo
 - o El esfuerzo indica la cantidad de horas lineales para desarrollar la user, pero el tamaño de una user es la cantidad de trabajo para terminar / producir una user
 - o El esfuerzo varía según habilidades, conocimientos, familiaridad con el dominio, etc
 - o El tamaño varía según las tareas que implican una user, es independiente de quien lo hace
- No hay que confundir esfuerzo con calendario
 - o Yo puedo decir que puedo tener un trabajo para el viernes pero entremedio pueden pasar cosas que evitan que se cumpla mi objetivo
 - o Un trabajo puede requerir un esfuerzo de x horas pero no se puede asegurar que para tal día va a estar terminado (consumir el 50% del tiempo estimado no es garantía que hice el 50% del trabajo)

STORY POINT

- Unidad de estimación que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una user en términos relativos respecto a otra user (me permite comparar users)
- Al medirse de forma relativa y no absoluta significa que por ejemplo una user de 8 puntos para un equipo no es lo mismo que una user de 8 puntos para otro equipo, pero lo importante es que un equipo sepa que una user de 5 puntos es mas chica que una de 8 puntos
- Varía mucho según el valor que se le asigna según si el equipo tiene o no experiencia
- Le da la idea de peso a cada user y permite mostrar cuan grande / compleja es (esa complejidad tiende a incrementarse exponencialmente y es por eso que usamos Fibonacci)
- No es una medida basada en tiempo (estimar basándose en el tiempo conduce errores)
- No se estiman todas las user, solo aquellas que están por entrar

VELOCIDAD

- Métrica de progreso de trabajo en equipo (mide el producto y sigue el principio ágil de que la mejor métrica del progreso es el software funcionando)
- Se calcula (no se estima) sumando el número de story points asignados a cada user que el equipo completa al finalizar cada sprint (implica que la user se desarrolló, testeó y aceptó por el PO)
- Si una user se completó parcialmente no se cuentan sus story points en esa iteración, lo que provoca que la velocidad del equipo baje en esa iteración, pero en la siguiente aumentará debido a que tienen varias story points casi terminadas que son de otro sprint (por eso debemos enfocarnos en el promedio de velocidades en sucesivas iteraciones que en cada iteración en específico)
- Siempre existen oscilaciones (un 20% es lo aceptable) ya que las personas viajan, toman licencias, etc no siempre tengo la misma capacidad de entrega en cada iteración
- No son métricas extrapolables a otros equipos, son de este equipo con este proyecto con este cliente el contexto lo es todo, si lo cambias, cambia toda la realidad
- Si sumamos el total de story points del backlog y la dividimos por la velocidad promedio del equipo podemos obtener la cantidad de iteraciones estimadas para finalizar el desarrollo (separamos la estimación del esfuerzo de la estimación de la duración cronológica, eliminando compromisos ciegos)
- Corrige errores de estimación inicial
- Me sirve para ver si voy logrando lo que pide el desarrollo sostenible

POKER ESTIMATION

- Es una técnica de estimación en ambientes ágiles
- El PO participa en la planificación pero no estima (suele ser el moderador)
- Es una mezcla entre juicio experto, analogía y desagregación
- Asume que las personas que desarrollar el producto deben ser aquellas que estimen
- Es mejor estimar en equipo ya que permite combinar experiencias y conocimientos de cada persona
- Dado a que la complejidad en el software incrementa exponencialmente, usamos la serie de Fibonacci como escala para determinar los puntos de historia
- **Escala**
 - o 0 → no tiene idea de su producto o funcionalidad (puede requerir un spike)
 - o 1 o ½ → funcionalidad pequeña
 - o 2 o 3 → funcionalidad pequeña a media
 - o 5 → funcionalidad media
 - o 8 → funcionalidad grande (es realizable pero no es lo óptimo, sería mejor dividir)
 - o 13 a 20 → son demasiado grandes y tienen un riesgo alto ¿xq no dividimos?
 - o 40 a 100 → es imposible ejecutar en una sprint

2,3,5 son estados óptimos, tienen el tamaño ideal y la granularidad perfecta para ejecutarse en un sprint (el 8 es el límite)
- **Procedimiento**
 - 1) Se definen una lista de users a discutir y se acuerda cual es la que tiene menor grado de incertidumbre, a esa la llamamos canónica y le asignamos 1 o 2 story points (es la más simple y sirve como referencia para comparar)
 - 2) Se lee una user a todo el equipo y el PO resuelve dudas
 - 3) Cada miembro elige una carta boca abajo para estimar el tamaño de la user (comparado con la canónica en base a tamaño, complejidad y riesgo) y se espera a que todos decidan
 - 4) Todos exponen sus cartas en simultaneo para evitar sesgos
 - 5) ¿Hay consenso total? **NO SE HACE POR MAYORIA, SE HACE POR CONSENSO**
 - Si → estimación guardada
 - No → el punto mas alto y el mas bajo justifican sus elecciones para que el resto del equipo pueda considerar cosas que quizás se les pasó por alto, ahora volvemos al punto 3 y repetimos hasta q haya consenso (si nunca llegamos a algo entonces usamos un promedio o agarramos la más repetida)

EMPIRISMO

- Agile es un enfoque empírico, basado en inspeccionar, adaptar y aprender continuamente
- Las estimaciones no son compromisos fijos, son pronósticos probabilísticos
- El trabajo se organiza en un ciclo iterativo de planear – hacer – inspeccionar – adaptar
- No hay planes definitivos, ni planificamos de una. Se va probando, midiendo y ajustando sobre la marcha
- Es clave medir la métrica de velocidad de cada iteración y ajustar para lograr el éxito

REESTIMACIÓN

- Volver a estimar una user es aconsejable solo cuando la opinión acerca del tamaño relativo de una user ha cambiado, debido a que en el proceso de estimación se han detectado otras users más grandes o pequeñas (esto se hace para mantener una consistencia de tamaños al estimar)
- No se reestima cuando no se detecta un progreso como el esperado, sino que se debe dejar que la velocidad a lo largo de las sucesivas iteraciones equilibre estas inexactitudes

PLANIFICACIÓN

- Es necesario en agile que el que planifique el proyecto sea capaz de analizar el contexto del proyecto cada cierto periodo para poder hacer aquellos ajustes que sean necesarios, para orientar los esfuerzos de manera correcta y poder lograr los objetivos planteados
- Planear lo es todo, los planes no son nada (lo importante es el proceso de planificar no los planes)
- **Se planifica en 3 niveles**
 - o Planificación de release
 - Se da al inicio del proyecto, se analiza y determina que users, épicas o temas serán desarrollados en una nueva entrega del producto
 - El plan se va actualizando mientras el proyecto transcurre, de forma que siempre refleja las expectativas actuales sobre que se incluye en la entrega
 - Una vez priorizada la user a implementar, se estima su tamaño mediante story points y según la velocidad de trabajo en equipo se calcula cuantos sprints se necesitan para terminar el proyecto o release
 - o Planificación de iteración
 - Se da al comienzo de cada iteración, donde el PO identifica las user de mayor prioridad a ser implementadas en la iteración, basándose en la iteración anterior
 - Se tiene en cuenta el tiempo y la velocidad de trabajo en equipo (ambas constantes)
 - o Planificación del día
 - Permite coordinar y sincronizar el trabajo del equipo, normalmente en dailys
 - Solo discutimos tareas a realizar en el día, haciendo foco en actividades que deben desarrollarse para lograr cumplir con una tarea específica

PARA QUE ESTIMAMOS

- **Determinar cuántas users puede comprometerse el equipo a terminar en una iteración**
 - o Si un equipo alcanza una velocidad sostenible en el tiempo (se mantiene su velocidad en distintas iteraciones) permite que el equipo obtenga previsibilidad, la cual ayuda a estimar en las siguientes iteraciones
 - o Si el equipo de trabajo es nuevo es difícil estimar cuantas user puede completar en una sprint y es muy probable que en las primeras le erre mucho pero se vayan adaptando
 - o Esto permite planificar de manera más realista, tomar decisiones sobre qué incluir en cada iteración y generar expectativas más confiables con los stakeholders.

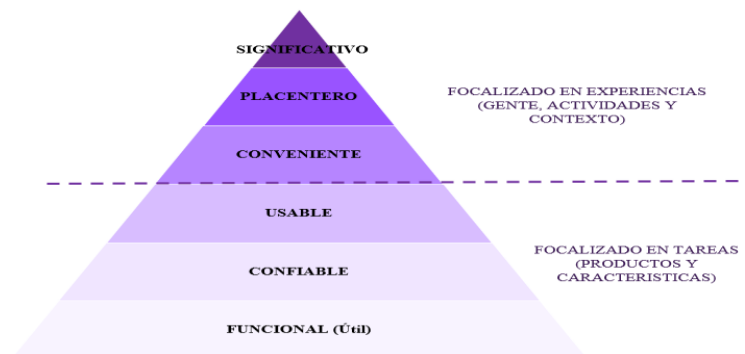
UNIDAD 6: GESTIÓN DE PRODUCTOS

PORQUE CREAMOS PRODUCTOS

- Para satisfacer a los clientes
- Para tener muchos usuarios logueados
- Para obtener mucho dinero
- Realizar una gran visión, cambiar al mundo

EVOLUCIÓN DE LOS PRODUCTOS DE SOFTWARE

- En la base de la pirámide tenemos aquellas cuestiones “básicas” que tienen que existir para que el software funcione como se lo espera
 - o **Funcional** → es por donde se empieza a construir el producto, se basa en que el software haga lo que tiene que hacer
 - o **Confiable** → si queremos avanzar, el software debe ser confiable (que los usuarios no corran peligro usando el producto y que los resultados sean certeros)
 - o **Usabilidad** → es poder lograr los objetivos usando el producto (el usuario debe sentir que el producto mejora su calidad de vida, que hace mejor su trabajo gracias al producto, esto no es lo mismo que productividad)
- Más allá de la línea que aparece en la pirámide, aparecen otros aspectos relacionados con otras cuestiones de la visión del producto que son difíciles de alcanzar, en general los productos que se construyen rara vez cruzan la línea y se quedan en la base de la pirámide



CICLO DE VIDA DE UN PRODUCTO

- En agile un producto no se construye todo de una vez, sino de forma iterativa e incremental
- Empezamos con una versión mínima del producto y desde ella se obtiene feedback por parte de los usuarios que nos permiten mejorar, corregir y agregar valor en cada iteración
- No es necesario entregar el 100% al inicio, porque no hay evidencia de que eso sea realmente útil
- Las fases son:
 - o Introducción
 - o Crecimiento
 - o Madurez
 - o Declive

EXPLICACIÓN MVP, MVF, MMF CON EL DINOSAURIO LEAN/AGILE

Paso uno

- Comprender que un producto nuevo tiene una hipótesis de valor único (UVP)



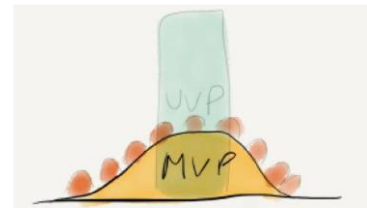
Paso dos

- Creamos un producto mínimo viable (MVP) para probar su hipótesis
- Esto se centra en su propuesta de valor única, pero normalmente también proporciona un poco de funciones de "Tablestakes" solo para asegurarse de que sea "viable" como producto.



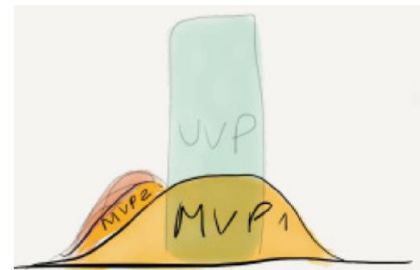
Paso tres

- El MVP es una hipótesis, podría ser lo suficientemente buena para encontrar un mercado o no
- En este paso le muestro la hipótesis a cada cliente para que interactúen y me den un feedback, pero puede suceder que cada cliente potencial me pida distintas funcionalidades (si esto pasa, significa que aún no está tan claro el mercado en el que el producto va a funcionar, por lo que debo seguir trabajando sobre la visión del producto)
- Otra cosa que puede pasar es que todos los clientes/usuarios que vean la MVP se sientan a gusto



Paso cuatro

- Si obtengo varias respuestas apuntando a x cosa, debemos revisar la hipótesis para ajustarla para que vaya hacia el lado que les interesa a los clientes (exigencias del mercado)
- Construimos un MVP2 centrado en la nueva hipótesis basada en el aprendizaje reciente de desarrollo de clientes generado por el anterior MVP
- Ahora con este nuevo MVP hacemos el proceso de retroalimentación (lo podemos hacer varias veces hasta tener un feedback exitoso)



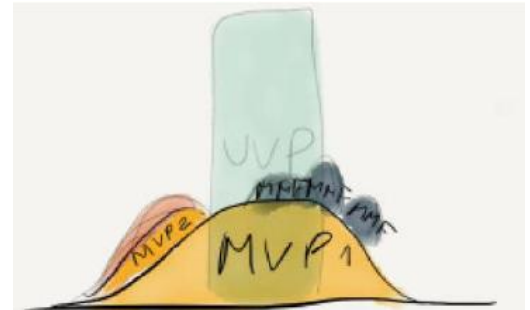
Paso cinco

- Si el MVP2 tiene éxito y muestra tracción real, el siguiente paso es escalar el crecimiento
- Para esto buscamos profundizar el valor para los usuarios actuales y atraer nuevos clientes más allá de los pioneros
- Mediante el feedback y la investigación se identifican oportunidades de crecimiento
 - o Ampliar las propuestas de valor (nuevas funcionalidades)
 - o Fortalecer el producto actual (hacerlo más sólido, completo y confiable)



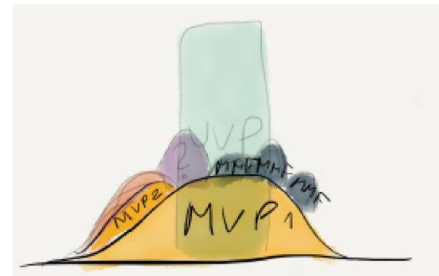
Paso seis

- En el caso de áreas con fuerte indicación de valor se pasa a definir un MMF (características mínimas comercializables)
- Se busca definir los elementos mínimos que deben estar presentes en un producto o servicio para que pueda traer crecimiento y valor al cliente (le sea rentable)
- Es importante construirlo en cuestiones mínimas para sacarlo rápidamente a la venta cuando el producto es atractivo y no está en el mercado (cuanto más tiempo pase, más riesgo corro de que mi idea sea implementada antes)
- La idea de aplicar el menor esfuerzo posible y no esfuerzos innecesarios antes de saber si el producto es comercializable o no (por esto que se dividen características grandes en MMF más pequeños)
- Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área.



Paso siete

- Si se desea esperar para ver si el primer MMF apesta entonces está de vuelta en la hipótesis.
- Ahora la hipótesis se centra en una característica en lugar del producto (hay un área con alto potencial pero también alta incertidumbre)



Paso siete

- La forma de afrontar el paso anterior es crear una función pionera, la MVF
- Es la característica mínima viable que se puede construir e implementar rápidamente (con los recursos mínimos) para probar la utilidad de la misma
- Es un features que buscamos que agregue valor agregado (buscamos aquella característica que puede hacer la diferencia y que hace que elijan mi producto sobre otro)
- Si el MVF resulta exitoso , puede desarrollar más MMF en esa área para tomar ventaja (si eso tiene sentido), sino se puede cambiar a otro enfoque hacia esa área de características o buscar una ruta de crecimiento alternativa
- El MVF es una versión mini del MVP.



Paso ocho

- El producto se cultiva en mercados inciertos al intentar varios MVP
- Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio / requisitos en las áreas en las que se está enfocando.
- Si bien los MVP / MMF / MVF son atómicos desde una perspectiva empresarial (no puede implementar y aprender de algo más pequeño) pueden ser bastante grandes en implementación
- El dinosaurio carpaccio se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a reducir el riesgo de ejecución / tecnología (normalmente denominadas users stories).
- Esas porciones más pequeñas pueden tener un valor comercial tangible o no (las que no tienen valor generalmente sirven para obtener una temprana retroalimentación)



ALGUNOS CONCEPTOS

- MVP (producto mínimo viable)

- Es una versión de un producto destinada a un subconjunto de potenciales clientes, que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre clientes, con el menor esfuerzo posible (tiempo y plata)
- Su objetivo no es vender, es validar si la hipótesis del producto a construir funciona o si hay que hacer cambios (busca ofrecer el producto a los clientes e iniciar un círculo de aprendizaje mediante la participación del usuario y la retroalimentación de si el producto cumple o no con las expectativas, guiando así el desarrollo futuro) gastando poco tiempo y plata
- Es lo mínimo que un producto debería tener para que los posibles usuarios del mismo lo puedan evaluar y decir si el producto les atrae o no (queremos ver si tiene el valor suficiente para que las personas estén dispuestas o no a comprarlo y demuestra suficiente beneficio futuro para retener a los primeros usuarios - comprar no es siempre algo económico -)
- Enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos
- Es más confiable ver lo que hace la gente con un producto a preguntarles que harían
- Más cercano a los prototipos que a una versión real funcionando de un producto

- MMF (característica mínima comerciable)

- Es la pieza más pequeña de funcionalidad que puede ser liberada aportando valor inmediato y crecimiento (tiene alta certeza comercial)
- Característica a pequeña escala que se puede construir e implementar rápidamente, usando recursos mínimos y para una población objetivo, para probar utilidad y adopción de la característica
- Tiene valor para la organización y para los usuarios
- Es una parte de un MMR o MMP (decimos que es una versión mini del MVP)
- Aporta al crecimiento inmediato y mejoran el time to market

- MMP (producto mínimo comerciable)

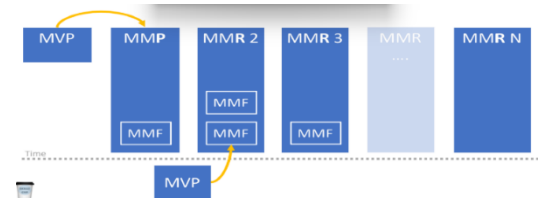
- Primer release de un MMR dirigido a primeros usuarios (early adopters)
- Es un producto listo para su lanzamiento en el mercado
- Focalizado en características claves que satisfarán a este grupo clave
- Es capaz de cubrir una amplia gama de funcionalidades, atendiendo a grandes bases de usuario e incluso podría ser una versión posterior de un MVP anterior (aunque probablemente aun necesite ajustes de UX UI)

- MMR (release mínimo comerciable)

- Release de un producto que tiene el conjunto de características más pequeño posible
- cuando un producto esta por salir al mercado decimos “che estas son las características mínimas que debe cumplir un release para soltarlo al mercado”)
- Es el incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales
- Un release es una versión de producto que tiene la madurez suficiente para salir al mercado
- MMP = MMR1

RELACION ENTRE MVP, MMF, MMP, MMR

- Partimos de una MVP (validar hipótesis) que va evolucionando hacia una MMP y aparecen MMF
- Cuando una MMP sale al mercado y tiene éxito, se considera el primer release real del producto.
- A partir de ahí, el producto evoluciona mediante nuevos releases, incorporando mejoras, ajustes y nuevas funcionalidades de forma continua (mientras el producto este en el mercado siempre va a requerir cambios)
- Si en algún momento surge una nueva idea y no se sabe si tendrá valor, se puede desarrollar un nuevo MVP para validarla. Según el resultado, esa funcionalidad se incorpora (o no) a futuros releases.
- Es clave salir al mercado lo antes posible, sin esperar a que el producto esté 100% completo o perfectamente especificado



MVF (CARACTERISTICA MÍNIMA VIABLE)

- Es una versión mini del MVP.
- Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica.
- Debe proporcionar un valor claro a los usuarios y ser fácil de usar.
- Requiere recursos mínimos, los estándares de calidad de la industria y la producción deben guiar el diseño y la confiabilidad.
- El grupo de usuarios ideal para un MVF está formado por los primeros en adoptarlo: clientes leales que ya han compartido conocimientos anteriormente o miembros de una junta asesora de clientes. Son usuarios flexibles, tolerantes y dispuestos a brindar retroalimentación para mejorar el producto
- Los resultados le ayudarán a tomar decisiones estratégicas sobre productos
- Son funciones pioneras en áreas de alta incertidumbre diseñadas para lograr aprendizaje continuo (no están completas xq no se validó si vale la pena agregarlo o no)

ERRORES COMUNES (MVP VS MMF O MMP)

- Confundir a un MVP (enfocada en aprendizaje y validar la hipótesis) con una MMF (característica comerciable misma) o con MMP (producto comercializable mínimo), enfocados en ganar
- El riesgo es entregar algo sin considerar si es lo correcto que satisface las necesidades del cliente.
- Enfatizar la parte mínima de MVP y no la parte viable (el MVP debe ser máximo Y viable a la vez). Si esto pasa, el producto entregado no es de calidad suficiente para proporcionar evaluación precisa de si los clientes utilizarán el producto (no es posible obtener retroalimentación)
- proporcionar una evaluación precisa de si los clientes utilizarán el producto.
- Hacer del MVP un producto final, es decir entregar lo que considera un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.

PRODUCTOS MÍNIMOS PARA LA GESTIÓN DE PRODUCTOS

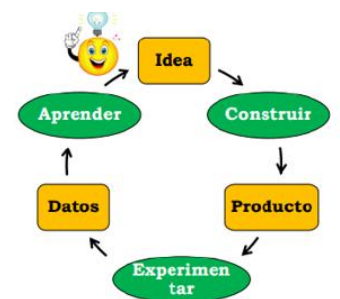
- **MVP** (producto mínimo viable)
- **MVF** (característica mínima viable)
- **MRF** (características mínimas del release)
- **MMF** (característica mínima comercializable)

VALOR VS DESPERDICIO

- ¿Cuáles de nuestros esfuerzos crean valor y cuáles son desperdicio?
 - o Todo aquello que provea beneficios reales a los clientes es el valor
 - o Cualquier otra cosa es un desperdicio
- La productividad de un Startup no puede medirse en términos de cuánto se construye cada día, por el contrario se debe medir en términos de averiguar la cosa correcta a construir cada día
- El MVP es lo que usamos para generar producto de valor y eliminar desperdicios (la idea es hacer ciclos cortos, realimentar aprender y determinar el producto correcto)

CICLO DE FEEDBACK O APRENDIZAJE VS DESPERDICIO

- Es un proceso iterativo el cual nos permite descubrir las necesidades que tiene el cliente y alinearlas metodológicamente
- Partimos de una hipótesis a validar (idea), luego construimos y obtenemos un producto, el cual se pone bajo consideración (lo prueban usuarios que usan/usarán el producto/servicio) y mediante los resultados podemos aprender y sacar conclusiones para comenzar un ciclo nuevamente
- Es más fácil pivotear el producto y volver al inicio desde el primer momento en el que el usuario no aprueba el producto (podemos equivocarnos)
- El éxito no es entregar un producto, sino que se trata de entregar un producto o característica que el cliente usará (la forma de hacerlo es alinear los esfuerzos continuamente hacia las necesidades reales de los clientes)



LA FASE DE CONSTRUIR EL MVP

- Al construir un MVP, su complejidad puede variar desde pruebas de humo extremadamente simples (como anuncios) hasta prototipos tempranos con características faltantes o problemas
- No tiene por qué ser un producto funcional, es cualquier cosa que deje mostrar mi idea
- La idea es tenerlo rápido y con el menor esfuerzo posible para así poder validarlo y retroalimentar la hipótesis para entrar en el ciclo de aprendizaje
- La fase de experimentar pasa a ser clave para la construcción de MVP

- ¿Cómo hacemos para minimizar el desperdicio y esfuerzo al construirlo?
 - o En caso de duda, simplificar
 - o Evitar la construcción y promesa excesiva
 - o Cualquier trabajo adicional, más allá de lo necesario para comenzar el ciclo, puede ser un desperdicio

DILEMA: LA AUDACIA CERO

- A menudo es más fácil recaudar dinero cuando tenemos cero ingresos, cero clientes y cero tracción a que cuando tenemos una pequeña cantidad de cada uno (cuando no hay nada para perder es cuando más te animas a arriesgarte, pero cuando ya tenemos clientes, estructuras, empleados, etc ahí siempre nos paramos a pensar en las restricciones)
- Cero invita a la imaginación, pero los números pequeños hacen preguntas sobre si los números grandes alguna vez se materializarán
- Este fenómeno (audacia de cero) crea un incentivo brutal y es que aplaza el lanzamiento de cualquier versión de un producto hasta que se esté seguro del éxito (para evitar cero clientes / cero ingresos)
- Posponer experimentar con la MVP trae resultados desafortunados como
 - o La cantidad de trabajo desperdiciado puede aumentar
 - o Se perderán los comentarios esenciales
 - o El riesgo de que su startup construya algo que nadie quiere puede aumentar
- Preferiría atraer capital de riesgo y potencialmente derrocharlo o preferiría atraer capital de riesgo y usarlo sabiamente?
- Debemos usar el MVP para experimentar (inicialmente, en silencio) con los primeros usuarios en el mercado, verificando mi hipótesis probando todos los elementos, comenzando por los más riesgosos.
- Esta audacia cero garantiza el aislamiento y aumenta el riesgo de fracaso masivo (el antídoto es la iteración continua)

SUPUESTOS DE SALTOS DE FE

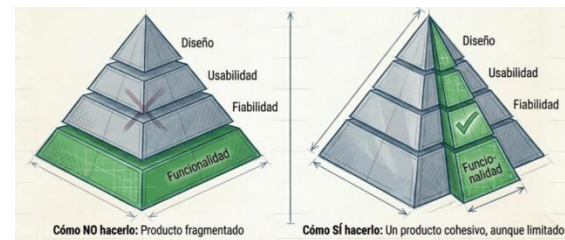
- Son los elementos más riesgosos del plan/concepto de un nuevo proyecto o startup (son las partes de la que todo depende, si están mal definidas TODO está mal)
- La mayoría de las personas no conocen una determinada solución (o incluso el problema), pero una vez experimentan la solución no pueden imaginar como vivirían sin ella (ósea que las personas no saben el valor de una solución hasta experimentarla – ejemplo uber-)
- Los dos supuestos son
 - o **Hipótesis del valor**
 - Prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es el producto correcto)
 - Una métrica de prueba puede ser la tasa de retención (ver si usan la app y vuelven o no)
 - o **Hipótesis de crecimiento**
 - Prueba cuantos nuevos clientes descubrirán el producto (ver si más personas desean probar el producto)
 - Una métrica de prueba puede ser la tasa de referencia

PREPARAR UN MVP

- Los pasos son
 - o Encontrar un nicho de mercado
 - o Cread un road map realista (definición de las características que tendrá el producto a cierta altura) en este paso
 - o Investigar la competencia (ver hasta donde llega y que fortalezas tiene)
 - o Pre-vender el MVP
 - o Testear las distintas suposiciones
 - o Asegurarse que el MVP resuelva el problema correcto
 - o Focalizarse en las funcionalidades principales

CARACTERISTICAS UN MVP

- Diseño
 - o Debe tener un diseño adecuado y una UX que deleite
 - o Hay que satisfacer el aspecto visual y de interacción
- Usabilidad
 - o Debe haber suficiente valor para el que esté dispuesto a usarlo/comprarlo
 - o Tiene que ser útil para el público objetivo
- Confiabilidad
 - o Involucra a que los early adoptes puedan confiar en la solución plenamente incluso aunque tenga poco tiempo en el mercado
- Funcionalidad
 - o Tiene las funciones necesarias para solucionar un problema en específico
 - o Satisface las demandas y permite evaluar las funciones a implementar más adelante



MATRIZ DE PRIORIZACIÓN PARA EL MVP



MLP (PRODUCTO MÍNIMO ADORABLE)

- Es la versión más simple de un producto que genera una conexión emocional o deleite en los usuarios (requiere investigar qué es lo que los clientes consideran como valioso, por eso lleva más tiempo que un MVP)
- Se centra en características que no solo son funcionales o útiles, sino que son lo suficientemente impactantes para deleitar a los usuarios (enfatisa la satisfacción del usuario y la diferenciación)
- Fomenta la lealtad y la adopción temprana
- Marca el nivel mínimo de diseño/desarrollo para q los clientes lo amen en lugar de tolerarlo
- La idea es crear un MVP que, además de proporcionar información al equipo de trabajo para el desarrollo futuro, apele a la emocionalidad del usuario para lograr lo más difícil: enamorallo

- Porque crearlo
 - o **Reputación de marca**
 - Si eres una gran empresa, cada producto que lances debe demostrar a los usuarios que te importan las características que necesitan. Si no, tus nuevos productos podrían parecer un intento de lucro que busca capitalizar a tu base de clientes actual.
 - o **Productos destacados**
 - Las startups que intentan competir con competidores bien financiados necesitan crear un producto mínimo atractivo que destaque en el mercado
 - Los MVP no son suficientes para revolucionar un mercado con soluciones consolidadas (el producto debe capturar la imaginación de inmediato)
 - o **Ciclo de retroalimentación del usuario.**
 - Al captar la atención de los usuarios desde el principio, podrás capitalizar su interés a medida que el MLP se desarrolla. Esto te proporciona un flujo fiable de retroalimentación del usuario que puedes aprovechar para la mejora continua.

COMO CREAR UN MLP

- **Sentir la curiosidad**
 - o No adivine que valoran los clientes, pregúnteles directamente
 - o ¿que le importa? ¿Qué problemas enfrenta? ¿Qué mejoraría su día?
- **Conocer el mercado**
 - o Comprenda el panorama en el que se encuentra antes de desarrollarlo y luego concéntrese en satisfacer las necesidades de sus clientes objetivo antes de expandirse a otros
- **Definir el propósito**
 - o Depure lo que aprenda sobre sus clientes y mercado para crear una visión de lo que desea lograr
- **Centrarse en el conjunto**
 - o Recuerde que su producto no es solo un conjunto de características, sino también todos los puntos de contacto con su empresa a lo largo de todo el recorrido del cliente
- **Construir para durar**
 - o Tome decisiones arquitectónicas con seguridad en respuesta a las necesidades o problemas de los clientes
 - o Confíe en el compromiso que ha asumido con su solución
- **Dedicar esfuerzo**
 - o No escatimes en gastos, invierta esfuerzo y los recursos necesarios para brindar un buen servicio a tus clientes desde el principio
- **Medir el cariño**
 - o Elige una forma de medir el cariño que despierta su product

CLAVES PARA DEFINIR UN MLP

- Entender y conocer el público objetivo
- Definir una propuesta que aporte valor a la necesidades y expectativas del usuario ¿en qué y cómo me diferencio del resto? ¿Qué valor añadimos al usuario? ¿Cuál es el aspecto novedoso?
- Resolver al menos una de las necesidades del usuario de manera excelente
- Sorprender al usuario añadiendo un factor sorpresa en la parte visual o funcional del producto

ELEMENTOS CLAVES DE UN MLP

ASPECTO	FUNCIONALIDAD	USABILIDAD	DELEITE
ROL	Resolver el problema del usuario	Asegurar facilidad de uso	Evocar emociones positivas
FOCO CLAVE	Aspectos core	Interface intuitiva	Interacciones intuitivas
BENEFICIO PRINCIPAL	Satisfacer necesidades básicas	Reducir fricción	Fomentar la lealtad
EJEMPLO	Apps de streaming de musica	App con navegación simple	App con playlist personalizadas

TABLAS COMPARATIVA DE ENFOQUES Y FOCOS

ASPECTO	ENTENDER LAS NECESIDADES DEL USUARIO	DISEÑO PARA EL DELEITE	DESARROLLO Y PRUEBA
ENFOQUE	Recopila info de los usuarios	Crea una experiencia atractiva	Construir y refinar el producto
FOCO CLAVE	Investigación, empatía	UI, impacto emocional	Coding, Testing de usuario
BENEFICIO PRINCIPAL	Informar el alcance del producto	Aumenta el amor del usuario	Asegura calidad
EJEMPLO	App user interviews	App fun UI animations	App beta testing

Spotify es un MLP porque no solo resuelve el acceso a música, sino que ofrece una experiencia altamente personalizada e intuitiva que genera valor emocional y fideliza al usuario.

BLOQUES DE CONSTRUCCIÓN DE UN MLP

- **Inspiración (ayudan a perseguir una vida con sentido)**
 - o Motivación
 - o Diversión
 - o Imagen positiva (halo)
- **Crecimiento**
 - o Confianza
 - o Credibilidad
 - o Escalabilidad
 - o Sostenibilidad
- **Utilidad (resuelve mis problemas)**
 - o Esperanza
 - o Satisfacción
 - o Cuidado

UNIDAD 7: COMPONENTES DEL PROYECTO DE DESARROLLO DESOFTWARE

PROCESO, PROYECTO Y PRODUCTO

- La ingeniería de software se basa en 4 dimensiones (proceso, proyecto, producto y personas)
- Un proyecto es llevado a cabo por personas motivadas (con roles definidos que les permite asumir responsabilidades claras) que implementan herramientas para automatizar los procesos y tienen como resultado un producto que puede ser un bien o un servicio



- **Proceso**
 - o Es una plantilla, una definición abstracta que se materializa a través de los proyectos, en donde se adapta a las necesidades concretas del mismo
 - o Se expresa en forma teórica lo que se debe hacer para hacer software y debo adaptarlo al proyecto (si es definido) o terminar de armarlo (si es empírico)
 - o Según el tipo de producto o servicio a desarrollar debemos definir un proceso diferente
 - o Toman como entrada requerimientos y mediante un conjunto de actividades (estructuradas y guiadas por un objetivo) obtienen un producto o servicio de software como salida
 - o Scrum, TDD, XP, kanvan, ATD y otros son framework, NO procesos ya que no están completos, son lineamientos y buenas prácticas de una parte o partes de un proceso empírico
- **Producto**
 - o Es el software en sí
 - o Software no es solo código ejecutable, es conocimiento empaquetado a distintos nivel de abstracción (diseño de BD, user, la bd en sí, manuales de configuración, etc) es cualquier cosa que salga como ejecución de una tarea de un proyecto
- **Proyecto**
 - o Unidad organizativa o de gestión que requiere personas, recursos y procesos para existir.
 - o Es un esfuerzo temporal cuyo objetivo es crear un producto de software único y de calidad
 - o El proceso se instancia en el proyecto, el proyecto toma el marco teórico del proceso y lo adapta a la realidad del proyecto (administra los recursos, coordina a las personas según disponibilidad y define que tareas son las necesarias para cumplir el objetivo)
 - o Un factor muy importante es la gente, la mayoría de las veces que se fracasa se deben por problemas en los requerimientos (en la elicitaciones, y lo que pasa ahí es un problema de comunicación entre equipo de desarrollo y los usuarios/clientes) no es un problema técnico, fallamos en la parte blanda

PLANIFICAR

- En algún punto debemos planificar (incluso los proyectos planificados fracasan, imagínate si no planificamos..)
- Lo importante es el acto de planificar, no el resultado (lo que sirve es pensar el objetivo, el alcance, el porque hacemos esto, si existen riesgos, que recursos usar, etc) todo eso es un trabajo valioso ya que de esto aprendemos (luego debemos dejar las decisiones documentadas)

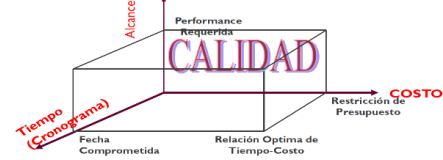
CARACTERISTICAS DEL PROYECTO

- **Duración limitada**
 - o El proyecto es temporario, tiene un inicio y un fin
 - o Al alcanzar el/los objetivos el proyecto termina y se liberan los recursos asociados
 - o Una línea de producción no es un proyecto
- **Son únicos**
 - o El resultado de un proyecto es único e irrepetible, por más parecidas o iguales que sean sus características (es erróneo decir que frente a mismas entradas obtengo mismas salidas)
 - o Todos los proyectos por más similares que sean, siempre tienen características que los hacen únicos
- **Orientado a un objetivo**
 - o Debemos tener un objetivo único (cada proyecto tiene un producto resultante distinto)
 - o Los proyectos están dirigidos a obtener resultados que dejen contento al cliente, y esto se refleja mediante objetivos (los cuales guían el proyecto)
 - o Los objetivos guían al proyecto y deben ser
 - **Claro** → todos los involucrados deben tener claro cuál es el objetivo
 - **No ambiguo** → deben ser lo suficientemente claros para guiar el desarrollo
 - **Alcanzable** → debe ser algo que se pueda lograr, participar en proyectos no alcanzables es participar de un proyecto destinado al fracaso
 - **Medible** → tengo que poder determinar cuánto avancé sobre el proyecto
- **Tienen tareas relacionadas entre sí basadas en esfuerzos y recursos para alcanzar el objetivo**
 - o Las tareas tienen relación entre si basadas en esfuerzos y recursos específicos (existen tareas que pueden hacerse al mismo tiempo, otras que requieren que x tarea se termine previamente, etc)
 - o Esta interrelación genera dependencias inevitables y eso hace que la gestión del producto sea una tarea compleja

ADMINISTRACIÓN DE PROYECTOS // PROJECT MANAGMENT // GESTION TRADICIONAL

- Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto y obtener como resultado el producto esperado
- Es la forma en la que administramos los recursos, las personas y el tiempo para obtener un producto o servicio de calidad que cumpla con lo que el cliente quiere en tiempo y forma
- Una buena gestión no garantiza el éxito del proyecto, pero una mala usualmente implica fallas
- La administración incluye:
 - o Identificar requerimientos
 - o Establecer objetivos claros y alcanzables
 - o Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders)
- Es una parte fundamental para que el proyecto sea exitoso y tiene dos grandes actividades
 - o Planificación
 - o Monitoreo y control

TRIPLE RESTRICCIÓN O TRIANGULO DE HIERRO



- Las variables de restricción de un proyecto son 3: tiempo, costo y alcance
- El balance de estos tres factores afecta directamente la calidad del proyecto
- El líder del proyecto es el encargado de balancear estas variables
- Un proyecto de alta calidad es aquel que entrega un producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado
- No podemos fijar de forma arbitraria las 3 variables ya que están tan relacionadas que modificar una de ellas seguramente implica cambios en las otras dos
- Nunca debemos negociar la calidad del producto, si podemos negociar las otras variables
- Siempre deben dejarnos alguna libre al menos xq sino no hay nada que hacer (no debería venir un cliente y decirme quiero x cosa en esta cantidad de días y con este presupuesto, xq no puedo hacer nada)
- **Alcance u objetivos ¿Qué trata de alcanzar?**
 - o Requerimientos que se incluyen en el proyecto y definen las funcionalidades que debe tener el producto a entregar en el cliente
 - o En tradicional es lo primero que se determina y se deja fijo en función de las necesidades del cliente (en agile se va definiendo, refinando y adaptando en base a retroalimentaciones)
- **Tiempo ¿Cuánto tiempo me llevaría?**
 - o Fecha calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente (en agiles el tiempo es fijo)
- **Costo ¿Cuánto debería costar?**
 - o Es el costo de todos los recursos implicados en el desarrollo del proyecto

En realidad el alcance está directamente relacionado con el tiempo y el costo (si aumentan la cantidad de funcionalidades –alcance- vamos a tener que usar más tiempo y dinero – costos).

El problema de esto es que en enfoques tradicionales, al tener procesos definidos (definir requerimientos al comienzo del proyecto) cualquier modificación o cambio en el alcance debería modificar el tiempo y presupuesto del proyecto pero en la realidad el cliente siempre quiere que se respete lo que se dijo en un primer momento (ignorando los añadidos) por eso que muchas veces se saltan etapas importantes o se sacrifica calidad y eso no debería ser así

ROLES DENTRO DE UN PROYECTO

- **Líder de proyecto o project manager**
 - o Es quien se ocupa de asignar todas las tareas de gestión que hacen que el equipo sea guiado a lograr el objetivo y maneja todas las relaciones con los stakeholders del proyecto
 - o En un enfoque tradicional toma decisiones en su totalidad sin incluir al equipo
 - o Por lo general no se recomienda que el PM sea también un perfil técnico (ya que se suelen priorizar las cosas técnicas y no las gestión y “nos quedamos sin gestión”)
 - o Este rol no existe en agilismo (los proyectos se autogestionan)
 - o Trabaja con las 3 variables de la triple restricción
 - o Requiere algunas habilidades como
 - Blandas → motivación, organización, innovación, empatía, comunicación, liderar, etc
 - Duras → relacionadas a los conocimientos del producto, técnicas y herramientas (resolver problemas, identidad administrativa, recompensar iniciativas y logros)
 - o Tiene tareas como: definir alcance del proyecto – identificar involucrados, recursos y presupuesto – definir las tareas, asignarlas y estimar los tiempos – identificar y evaluar riesgos – preparar planes de contingencia – producir reportes de estado - seguimiento de lo planificado – administrar recursos- etc



- **Equipo de proyecto**
 - o Grupo de personas comprometidas en alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables
 - o Tienen sentido de responsabilidad como una unidad
 - o Todo el equipo apunta a perseguir el mismo objetivo, cada persona tiene habilidades, conocimientos y diversas personalidades que permiten que las personas se complementen y hacer la diferencia entre trabajar individualmente o en equipo
 - o Busca trabajar en conjunto con espíritu de grupo, convirtiéndose en un grupo cohesivo en el que prima la sinergia
 - o Se suelen elegir grupos pequeños para mejorar y fomentar la comunicación (poder trabajar en equipo sin problemas)
 - o Hay que evitar cambiar o integrar miembros constantemente para lograr un equipo cohesivo
- **Stakeholders**
 - o Son los interesados del proyecto (equipo del proyecto, equipo de dirección, líder del proyecto, patrocinador, etc)

PLAN DE PROYECTO

- Es como una hoja de ruta de un proyecto, en él se documentan todas las decisiones que se toman a lo largo del desarrollo (procesos, división de tareas, asignar responsabilidades, equipos, etc)
- Provee la dirección esencial pero exige adaptación en tiempo real ante las variables constantemente cambiantes (lo vamos corrigiendo mientras avanza el proceso)
- **Esto es solo para tradicionales, en agil usamos el product backlog, sprint backlog, plan release???**
- **Se documenta**
 - o ¿QUÉ ES LO QUE HACEMOS? Alcance del proyecto
 - o ¿CUÁNDO LO HACEMOS? Calendarización
 - o ¿CÓMO LO HACEMOS? Recursos y decisiones disponibles
 - o ¿QUIÉN LO VA A HACER? Asignación de tareas

LA PLANIFICACIÓN DE PROYECTOS DE SOFTWARE IMPLICA UNA SERIE DE ACTIVIDADES

- **Definición del Alcance del Proyecto**
 - o Responde al qué se está desarrollando
 - o Ambos pueden modificarse a lo largo del tiempo (no implica modificar el alcance del producto)
 - o Si el producto a construir es grande las tareas del proyecto llevarán más tiempo y recursos (primero debo definir el producto para poder pasar al proyecto)

ALCANCE DE UN PRODUCTO	ALCANCE DEL PROYECTO
Todas las funcionalidades que deben incluirse en un producto o servicio para satisfacer los requerimientos del cliente	Todo el trabajo y solo el trabajo que se debe hacerse para entregar un producto o servicio con todas las funciones especificadas
Su cumplimiento se mide contra la ERS (se mide en requerimientos)	Su cumplimiento se mide contra el plan del proyecto (se mide en tareas)
Por ejemplo un caso de uso, users, requerimientos, etc	Por ejemplo testear, desarrollar un componente, etc

El ciclo de vida del proyecto es algo más pequeño y acotado, un producto tiene varios proyectos a lo largo de su vida (uno para la primera versión luego los que hagan falta para que vaya creciendo)

- Definición de Proceso y Ciclo de Vida

- Al iniciar el proyecto debo definir qué proceso y ciclo de vida quiero usar
- El proceso de desarrollo es el conjunto de actividades que requiero ejecutar para construir el producto de software (responde al cómo se va a desarrollar el proyecto)
- El ciclo es como ejecuto esas actividades (responde a cuanto de cada tarea hacer y en qué momento hacerlo)
- Al definir el proceso Debemos tener en cuenta complejidad del software a desarrollar (alcances y objetivos) y la información del contexto (objetivos, alcances, recursos y personas disponibles, etc)

- Estimación

- Al estimar en un proyecto tradicional, debemos definirlo antes de hacerlo
- Debemos buscar la manera de poder estimar todas estas características (lo haremos con cierto nivel de incertidumbre): tamaño - esfuerzo – calendario – costo – recursos críticos
- Los recursos críticos son cosas especiales de un proyecto x ejemplo un sensor de humo

- Gestión de Riesgos **SE EJECUTA ITERATIVAMENTE A LO LARGO DE TODAS LAS FASES DEL PROYECTO**

- **Riesgo** → problema esperando para suceder que tiene una probabilidad de ocurrencia (si ya ocurrió no es un riesgo) y que podría comprometer el éxito del proyecto si llegase a suceder
- Para medir los riesgos usamos la exposición al riesgo que surge de multiplicar la probabilidad de que ocurra el riesgo (probabilidad de ocurrencia) por el impacto que el mismo causaría si sucedería (impacto). Este valor me permite gestionar los riesgos para disminuir la exposición
- A medida que avance el proyecto los riesgos pueden desaparecer, aparecer nuevos o cambiar sus probabilidades
- Los riesgos se pueden negar, gestionar reactivamente (esperar que ocurra y ver que hacer) o proactivamente (trabajo constante sobre el riesgo mediante planes de mitigación y contingencia)
- Ejemplos de riesgo: cambios de requerimientos, abandono de integrantes, falta de capacidad
- **Existen riesgos de**
 - Proyecto (alteran el calendario) ejemplo se va un diseñador experimentado
 - Producto (afecta el rendimiento del software a desarrollar) ejemplo un producto que falla
 - Negocio (afectan la organización que desarrolla/adquiere el software) ejemplo nueva competencia
- **Pasos**

▪ Identificación de riesgo

- Identificamos los riesgos que suponen mayor amenaza (es imposible gestionar todos los riesgos) consultando la BD corporativa

▪ Análisis de riesgos

- Se analiza la probabilidad de ocurrencia y el impacto de cada riesgo
- Pueden existir riesgos con mucho impacto pero poca probabilidad de ocurrir
- Es muy importante definir cuales atender para hacerles un seguimiento (el criterio dependerá de la experiencia de quien analice los riesgos)



- **Planeación de riesgo**

- Para cada riesgo analizado se planifican estrategias para manejarlos (puedo disminuir la probabilidad el impacto o ambos)
- Estas estrategias son que acciones debemos realizar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la ocurrencia que representa el riesgo

- **Monitorización de riesgo y control**

- Proceso constante que permite determinar que las suposiciones sobre los riesgos de producto, proceso y organización no han cambiado
- Permite revalorar al riesgo en termino de posibles variaciones de su probabilidad e impacto

- **Retroalimentación**

- Esto es un ciclo que permite que aprendamos y generar una base de conocimiento corporativo (nos retroalimentamos para futuros proyectos)

- **Asignación de Recursos**

- Asignar responsabilidades a las personas permite asignar roles a los integrantes del equipo de trabajo
- Una persona puede tener más de un rol dentro de un mismo proyecto y un rol puede ser ejecutado por más de una persona (esto puede pasar por bajos presupuestos, proyectos no muy grandes)
- Es muy importante la selección del personal de un proyecto, ya que las capacidades de los integrantes del equipo afectan la calidad del software y el correcto desarrollo del proyecto ya que el software es una actividad humano-intensiva

- **Programación de Proyectos**

- Definimos en detalle cada tarea que debe cumplirse en el desarrollo
- Se toma como base lo definido en la estimación del calendario previo pero se refina al máximo detalle cada actividad
- Detallar tareas se refiere a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando y cuanto esfuerzo le lleva

- **Definición de Controles**

- El PM debe determinar si se está cumpliendo o no lo definido en el plan del proyecto, para esto se basa en el plan de proyecto y las métricas (voy comparando lo planificado vs real)
- Un proyecto se atrasa de un día a la vez, esto significa que si puedo corregir los desvíos de mi proyecto en el momento adecuado, estoy a tiempo de poder cumplir mi objetivo (si veo desvíos en el mes numero 5 de un proyecto de 6 meses estoy al horno)
- Que algo vaya “bien” no siempre es bueno: puede deberse a una sobrestimación (planificaste más tiempo/esfuerzo del necesario y por eso parece que vas adelantado)
- Se definen reuniones periódicas, reportes o informes necesarios

- **Definición de Métricas (PARA AMBIENTES TRADICIONALES)**

- Medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo (ver si estamos alineados o desviados respecto a lo planificado)
- Es bueno tener métricas, ya que son números y los números son cuantitativos y objetivos (no decimos nos falta poco, que es una percepción subjetivo, decimos un número concreto)
- Al planificar el proyecto debemos definir de forma clara y no ambigua las métricas y las formas de calcularlas
- Una métrica sin un fin particular no tiene sentido, deben proporcionar un aumento de info
- En agile el mayor indicador del avance de un producto son los incrementos que se entregan en cada iteración (acá las métricas miden y exponen los avances)
- **Métricas básicas**
 - Tamaño - Esfuerzo- Tiempo (calendario) – Defectos
 - Se deben contar efectos por severidad (dependiendo el tipo), no contar en masa
- **¿Cada cuanto tomar métricas?**
 - Tomamos métricas en base a decisiones sobre el tiempo que va a realizarse el proyecto (si el proyecto dura 3 años no vamos a medir diariamente ya que vamos a gastar muchos recursos sin un fin; tampoco debemos tomar cada 1 mes si el proyecto es de dos meses ya que no llego a corregir un desvío)
- **Algo más de métricas**
 - Si una métrica nos da siempre bien no tiene más sentido mirarla (estoy analizando algo que mi proceso ya tiene resuelto)
 - Busco minimizar los esfuerzos que requieren tomar las métricas, según las restricciones del proyecto (tiempo, recursos y alcance)
 - Lo mas importante de tomar métricas es para monitoreo y control (ver si voy bien)
 - Nunca deben usarse para medir el desempeño de las personas
 - Hay que tratar de mantenerlo simple y que sea fácil de calcular
- **Pueden clasificarse en**

Métricas de proceso	Métricas de proyecto	Métricas de producto
Permiten saber, independientemente de un proyecto en particular, si una organización está trabajado bien o mal (lo despersonalizamos de un proyecto y proceso en particular)	Permiten saber si un proyecto de software en ejecución se está cumpliendo según lo planificado o no (y ahí calibro) Mucha relación con triple restricción	Mide cuestiones/características que tienen relación directa con el producto que estamos construyendo
	Son métricas de esfuerzo y tiempo calendario	Son métricas de defectos o tamaños

Las métricas de proyecto se consolidan para crear métricas de proceso públicas para toda la organización del software

- **Proceso vs proyecto (ejemplo)**

- Proceso → q desviación hay entre lo estimado en esfuerzo y lo ejecutado realment
- Proyecto → porcentaje de lo estimado vs ejecutado en el proyecto en cuanto esfuerzo

- **Producto vs proceso (ejemplo)**

- Producto → este producto tiene 3 defectos cada modulo
- Proceso → mi organización tiene índice de defectos grave de 1 en un millón de líneas de código sin comentar

DESARROLLO DE SOFTWARE

- El desarrollo no termina cuando creo el producto sino que tengo un primer ciclo de desarrollo (obtengo el desarrollo de software) y a lo largo de la vida de ese producto vamos obteniendo nuevas versiones que son un incremento a la funcionalidad del producto

PLANES DE SOPORTE

- Planes de testing – mantenimiento – subcontrataciones – calidad – capacitaciones – iteraciones

ALGUNAS COMPLEJIDADES AL ADMINISTRAR PROYECTOS DE SOFTWARE

- El producto es intangible, no se puede ver ni tocar el producto que se obtiene al ejecutar el proyecto esto dificulta poder medir el progreso del proyecto
- Los proyectos de software suelen ser excepcionales, los conocimientos aprendidos en un proyecto de software son difícilmente extrapolables a otro proyecto
- Los procesos de desarrollo son variables y específicos de la organización (a pesar del intento de estandarizar los procesos de desarrollo, la realidad es que no se puede predecir de manera confiable que proceso es el adecuado para adoptar en el proyecto)

CAUSAS DE ÉXITO Y FRACASO

FACTORES PARA EL ÉXITO	FACTORES PARA EL FRACASO
<u>Monitoreo y feedback</u> Monitorear constantemente y corregir cuando sea necesario para evitar desviaciones mayores	No hay seguimiento del plan de proyecto
<u>Comunicación constante involucrando al usuario</u> Entre todos (desarrolladores, líder de proyecto, clientes, stakeholders)	Falta de involucramiento del usuario
<u>Misión/objetivo claro</u> Nos permite saber hacia dónde vamos	Fallas al definir el problema original y plan de proyecto pobre en detalles
Personal capacitado y competente	Las estimaciones se basaron en supuestos sin consultar datos históricos
Planificación adecuada y realista	Planificación de recursos inadecuada y basada en datos insuficientes
Apoyo de la gerencia	La planificación la hizo el grupo de planificadores
Hitos intermedios	Nadie estaba a cargo
Requerimientos correctamente enunciados (con una participación del usuario)	Falla en Elicitación de requerimientos (80%)
	Requerimientos cambiantes
	Expectativas poco realistas
	Falta de apoyo de la gerencia

UNIDAD 8: SCRUM

QUE ES SCRUM

- Marco de trabajo liviano y simple que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptables para problemas complejos
- En Scrum requerimos un scrum master para fomentar un entorno donde
 - o Un PO ordena el trabajo de un problema complejo en un product backlog
 - o El equipo convierte una selección del trabajo en un incremento de valor durante un sprint
 - o El equipo de scrum y sus partes interesadas inspeccionan los resultados y realizan ajustes necesarios para el próximo sprint
 - o Se repite
- Es un marco deliberadamente incompleto, define solo lo necesario para poder implementar la teoría de scrum y son las personas que lo utilizan las que definen como implementar ciertas cosas (no da instrucciones detalladas, guían relaciones e interacciones)
- Se basa en el empirismo (experiencia) y lean (reducir los desperdicios, centrándose en lo esencial)
- Tiene un enfoque iterativo incremental que permite optimizar la previsibilidad y controlar el riesgo
- Involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.
- Permite emplear diversos procesos, técnicas y métodos

PILARES

- Scrum tiene 3 pilares: transparencia, inspección y adaptación (son pilares del empirismo)
 - o **Transparencia**
 - Las decisiones importantes se basan en el estado de percibido de sus tres artefactos
 - o **Inspección**
 - Scrum proporciona cadencia en forma de cinco eventos que permiten ayudar con la inspección
 - No solo se revisa progreso, también los artefactos de scrum
 - o **Adaptación**
 - Se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse
 - Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Todo esto está definido más arriba, acá agrego solo cositas extras para las definiciones anteriores

VALORES DE SCRUM

- El éxito de Scrum depende de que las personas sean más competentes en vivir cinco valores: compromiso, enfoque, apertura, respeto y coraje (se van aprendiendo mientras usamos scrum)
- Los miembros en scrum tienen el valor de hacer lo correcto y trabajar en problemas complejos
- Estos valores le dan dirección al equipo de scrum con respecto a su trabajo, acciones y comportamiento (es por eso que la forma de usar scrum y las decisiones y medidas tomadas deben reforzar estos valores)

QUE ES UN EQUIPO DE SCRUM

- Unidad fundamental de scrum
- Unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del producto
- Son equipos
 - o Multifuncionales → todos los miembros tienen habilidades necesarias para crear valor
 - o Autogestionados → los equipos deciden por si mismos quien hace qué, cuando y como
 - o Lo suficientemente pequeños para permanecer ágil y lo suficientemente grandes como para completar un trabajo significativo en un sprint (como máximo 10 personas)
 - o Más pequeño = mejor comunicación = más productividad
- Si el equipo se vuelve demasiado grande debería reorganizarse en varios equipos cohesionados en el que todos comparten el mismo objetivo de producto, trabajo pendiente -product backlog- y PO
- Es responsable de todas las actividades relacionadas con los productos (colaboración, verificación, mantenimiento, operación, experimentación, investigación y desarrollo, y cualquier otra cosa que pueda ser necesaria)
- Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint
- Trabajar en Sprints a un ritmo sostenible mejora el enfoque y la consistencia del equipo de Scrum.

INTEGRANTES DE UN EQUIPO DE SCRUM

- **Desarrolladores**
 - o Personas del equipo que se comprometen a crea cualquier aspecto de un incremento útil (funcional) en cada Sprint
 - o El PO y el SM pueden ser desarrolladores
 - o No confundir con los que desarrollan!! El nombre es troll
 - o Tienen amplias habilidades específicas necesarias, pero siempre son responsables de
 - Crear un plan para el Sprint, el Sprint Backlog
 - Inculcar la calidad adhiriéndose a DoD
 - Adaptar su plan cada día hacia el Objetivo Sprint
 - Responsabilizarse mutuamente como profesionales
- **Product owner**
 - o Responsable de maximizar el valor del producto resultante del trabajo del equipo en scrum (la forma de hacerlo varía entre organizaciones, equipos scrum e individuos)
 - o Gestiona pila del producto (Product Backlog):
 - Desarrollar y comunicar explícitamente el Objetivo del Producto
 - Creación y comunicación clara de elementos de trabajo pendiente del producto
 - Pedido de artículos de trabajo pendiente del producto
 - Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.
 - o Esas tareas las puede hacer el o delegarlas (PERO LA RESPONSABILIDAD SIEMPRE ES DE ÉL) y para que tengan éxito toda la organización debe respetar sus decisiones
 - o Es una persona, no un comité
 - o Puede representar las necesidades de muchas partes interesadas en el trabajo pendiente del producto (si alguien quiere cambiar el trabajo pendiente puede dialogarlo con el PO)

- Scrum master

- Es el responsable de establecer de una manera eficiente scrum en el team tal cual como se define en la guía de scrum
- Para conseguirlo lo que hace es ayudar a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización.
- Son verdaderos líderes que sirven al equipo Scrum y a toda la organización.
- El Scrum Master sirve al equipo de Scrum de varias maneras, incluyendo:
 - Capacitar a los miembros del equipo en autogestión y multifuncionalidad
 - Ayudar al equipo de Scrum a centrarse en la creación de incrementos de alto valor que cumplan con la definición de hecho
 - Promover la eliminación de los impedimentos para el progreso del equipo Scrum
 - Asegurar de que todos los eventos de Scrum se lleven a cabo, sean positivos, productivos y que se respete el tiempo establecido (time-box) para cada uno de ellos.
- El Scrum Master sirve al Propietario del Producto (Product Owner) de varias maneras:
 - Ayudar a encontrar técnicas para una definición eficaz de los objetivos del producto y la gestión de los retrasos en el producto
 - Ayudar al equipo de Scrum a comprender la necesidad de elementos de trabajo pendiente de productos claros y concisos;
 - Ayudar a establecer la planificación empírica de productos para un entorno complejo
 - Facilitar la colaboración de las partes interesadas según sea solicitado o necesario.
- El Scrum Master sirve a la organización de varias maneras, incluyendo
 - Liderar, capacitar y mentorizar a la organización en su adopción de Scrum
 - Planificar y asesorar sobre la implementación de Scrum dentro de la organización;
 - Ayudar a las personas y a las partes interesadas a comprender y promulgar un enfoque empírico para el trabajo complejo
 - Eliminar las barreras entre las partes interesadas y los equipos de Scrum.

EVENTOS EN SCRUM

- El Sprint es un contenedor para todos los eventos de scrum
- Cada evento en Scrum es una oportunidad para inspeccionar y adaptar los artefactos de Scrum y están diseñados para permitir la transparencia necesaria (si no se realizan los eventos según lo prescrito, se pierden oportunidades de inspeccionar y adaptarse)
- Se usan para crear regularidad y minimizar las necesidades de reuniones no definidas en scrum
- Todos los eventos se llevan a cabo al mismo tiempo y lugar para reducir la complejidad
- **SPRINT**
 - Es donde las ideas se convierten en valor, aquí sucede todo el trabajo necesario para alcanzar el objetivo del producto (cada uno de los eventos)
 - Tienen duración fija de más o menos 1 mes para crear consistencia (apenas termina un sprint comienza inmediatamente el otro)
 - Durante un sprint
 - La calidad no disminuye
 - No se hacen cambios que pongan en peligro el objetivo del sprint
 - El trabajo pendiente del producto (product backlog) se refina según sea necesario
 - El alcance se puede clarificar y renegociar con el PO mientras aprendemos más
 - Si el objetivo del sprint se vuelve obsoleto el PO puede cancelar el sprint

- Permiten la previsibilidad al garantizar inspección y adaptación del progreso hacia un objetivo del producto, como mínimo, una vez al mes en el calendario
- Cuando el horizonte de un Sprint es demasiado largo, el Objetivo de Sprint puede volverse obsoleto, la complejidad puede aumentar y el riesgo puede aumentar.
- Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un período de tiempo más pequeño.
- Cada Sprint puede considerarse un proyecto corto.
- Existen varias prácticas para pronosticar el progreso (las vemos mas abajo)

LOS EVENTOS EN SCRUM SON

1) Planificación de sprint / sprint planning

- Inicia el sprint estableciendo el trabajo a realizar en el mismo
- Se crea mediante el trabajo colectivo de todo el equipo de Scrum (pueden invitarse gente extra para asesoramiento)
- El PO se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importantes y cómo se asignan al objetivo del producto
- Dura 8 horas para un sprint de un mes (para sprints mas chicos dura 5 horas)
- Se abordan temas como
 - ¿Por qué este sprint es valioso? **CUMPLIR CON EL OBJETIVO DEL SPRINT**
 - El PO propone cómo el producto podría aumentar su valor y utilidad en el sprint actual
 - Luego el equipo de scrum colabora para definir un objetivo del sprint que responda la pregunta para las partes iniciales
 - Antes que finalice esta etapa debe finalizarse el objetivo del sprint
 - ¿Qué se puede hacer en este Sprint? **QUE CARACTERISTICAS INCLUIR**
 - Los desarrolladores seleccionan los elementos del PB a incluir en el sprint actual (seleccionar cuanto puede ser un desafío, pero a medida que los desarrolladores tienen más experiencia van a estar más seguros de sus pronósticos)
 - El equipo scrum puede refinar estos elementos durante este proceso
 - ¿Cómo se realizará el trabajo elegido? **DESAGREGAR TÉCNICAS**
 - Para cada elemento de trabajo pendiente de producto seleccionado (Product Backload), los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con DoD
 - Normalmente esto se hace mediante a descomposición de elementos de trabajo pendiente del producto en ítems más pequeños que puedan realizarse en un día o menos (la forma de hacerlo la deciden los desarrolladores)

Siempre que hablamos del **trabajo pendiente de sprint o sprint backlog** nos referimos al objetivo de sprint + elementos de trabajo pendiente de producto seleccionado para el sprint + plan de entregarlos

Entrada (objetivo del producto, duración, capacidad del equipo, velocidad y restricciones)

Actividades (estimar cuantas features incluir dentro de un sprint según las entradas)

Salida(sprint backlog con las features del PB que decimos incluir en el sprint, y la descomposiciones de esas users (o no) en tareas a ejecutar)

2) Scrum diario (Daily scrum)

- Su objetivo es inspeccionar el progreso hacia el objetivo del sprint y adaptar el spring backlog según sea necesario, ajustando el próximo trabajo planeado
- Es un evento de 15 minutos máximo que se lleva siempre en el mismo tiempo y lugar todo los días laborables del sprint para que los desarrolladores del equipo (Un PO o SM pueden ser desarrolladores si participan del trabajo pendiente de sprint)
- Mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones.
- Puede se cualquier estructura o técnica deseada, mientras que se respete el objetivo y se produzca un plan para el día siguiente

3) Revision del sprint (sprint review)

- Su objetivo es inspeccionar el resultado del sprint y determinar futuras adaptaciones
- El equipo scrum presenta los resultados de su trabajo a las partes interesadas clave y discuten el progreso hacia el objetivo de producto
- No es una simple presentación, es una sesión de trabajo
- El scrum y las partes interesadas revisan que se logró y que cambió en su entorno y en base a eso los asistentes colaboran en qué hacer a continuación
- Es el penúltimo evento del sprint y se usan máximo 4 horas para sprint de un mes (si es más corto suele durar menos)
- El trabajo pendiente del producto se puede ajustar para satisfacer nuevas oportunidades

4) Retrospectiva del sprint (sprint retrospective)

- Su propósito es planificar formas de aumentar la calidad y la eficacia.
- El equipo de Scrum:
 - Inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y DOD (los elementos inspeccionados a menudo varían según el dominio del trabajo)
 - Las suposiciones que los desviaron se identifican y exploran sus orígenes
 - Analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no) resueltos.
 - Identifica los cambios más útiles para mejorar su eficacia (se agregan las mejoras mas impactantes lo antes posible)
- Esta actividad concluye el Sprint y se le da un máximo de 3 horas en un mes (en sprints más cortas puede durar menos)

5) Refinamiento del product backlog

- Es una actividad continua por eso está en términos relativos (10% DEL TIEMPO DEL SPRINT)

ARTEFACTOS DE SCRUM

- Representan trabajo y valor
- Cada artefacto contiene un compromiso para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso (existen para reforzar el empirismo y los valores de scrum para el equipo y partes interesadas)
- Buscan maximizar la transparencia de la información clave, permitiendo que cada uno de los que los inspecciona tenga la misma información y pueda adaptar de manera efectiva

LOS ARTEFACTOS DE SCRUM SON

- **Pila del producto (product backlog) VA PRODUCTO, NO TAREAS**
 - Es el trabajo pendiente del producto, organizado como una lista emergente (variante) y ordenada por prioridad de lo que se necesita para mejorar el producto
 - Es la única fuente de trabajo emprendida por el equipo Scrum (no hay tareas por afuera)
 - Los elementos de trabajo pendiente de producto que pueden hacer un equipo de scrum dentro de un sprint se consideran listos para su selección en la planificación del sprint
 - Por lo general el Product Backlog se vuelve claro y entendible recién después de refinarlo
 - ¿Qué es refinar?
 - Descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos
 - El tamaño se encarga de definirlo los desarrolladores (el PO puede ayudarlos a entender y seleccionar mejores alternativas pero NO es el que da el tamaño)
 - **Su compromiso es el objetivo del producto**
 - Describe un estado futuro del producto que puede servir como objetivo para el equipo scrum contra el cual planificar
 - Se encuentra en el Product backload
 - Es el objetivo a largo plazo para el equipo de scrum
 - Se debe abandonar o cumplir un objetivo antes de asumir el siguiente

- **Pila del sprint (sprint backlog) SI HAY ACÁ ESTAN LAS TAREAS**
 - El trabajo pendiente de sprint se compone de
 - Objetivo del sprint (por qué se hace)
 - Elementos de trabajo pendiente de producto seleccionados para el sprint (qué hacer)
 - Plan de trabajo (cómo se va a hacer)
 - El trabajo pendiente de un sprint es un plan por y para los desarrolladores (es una imagen muy visible en tiempo real del trabajo que los desarrolladores planean realizar durante el sprint para lograr el objetivo del sprint)
 - Se actualiza a lo largo del sprint a medida que aprenda más (no es rígido)
 - Debe tener suficientes detalles para poder inspeccionar su progreso en las dailys
 - **Su compromiso es objetivo de la sprint**
 - Es el único objetivo de la sprint
 - Es un compromiso de los desarrolladores y proporciona flexibilidad (dice que hay que hacer pero no como)
 - Direcciona al equipo para que todos trabajen hacia un mismo objetivo y no en iniciativas separadas
 - Se lo crea en el sprint planning y luego se lo agrega al trabajo pendiente del sprint
 - A medida que los desarrolladores trabajan durante el sprint tienen en cuenta el objetivo del mismo, si el trabajo resulta ser diferente de lo que esperaban, charlan con el PO para negociar el alcance del trabajo pendiente del sprint dentro del sprint sin afectar al objetivo del sprint (puedo hacer cambios sin afectar objetivo del sprint)

- Incremento

- Es un paso hacia el objetivo del producto
- Cada incremento es aditivo a todos los incrementos anteriores y verificado a fondo (me aseguro que todos los incrementos funcionen juntos) ya que para proporcionar valor el incremento debe ser utilizable
- En un sprint pueden surgir varios y la suma de todos ellos se presenta en la revisión del sprint (aunque un incremento puede entregarse a los interesados antes del final del sprint)
- Para que un trabajo sea parte de un incremento debe cumplir con la DoD
- **El compromiso es la definición de hecho (DoD)**
 - Es una descripción formal del estado del incremento cuando cumple con las medidas de calidad requeridas para el producto
 - En el momento en que un elemento de trabajo pendiente de producto cumple con la Definición de hecho se crea un incremento
 - DoD crea transparencia al proporcionar una comprensión compartida de qué trabajo se completó como parte del incremento
 - Si un elemento de trabajo pendiente de producto no cumple con DoD no se puede liberar ni presentar en la revisión de sprint (debe volver al trabajo pendiente del producto para su consideración futura)

TIMEBOX

- Esto existe para no perder más tiempo del necesario en las distintas áreas y aprender a negociar en términos del alcance del producto y no del tiempo o los costos NUNCA NEGOCIO CALIDAD
- Si no se llega al final de un sprint en lugar de extender el tiempo, se entrega menos
- De esta manera nos volvemos más confiable al mantener un ritmo de trabajo sostenible
- Todas las ceremonias tienen timebox (mas arriba las especifiqué en c/u) scrum me da un estimativo de cuanto debería durar cada una pero cada equipo puede definirlo según lo necesite

ALGUNAS COSAS

- Al concepto de DoD de las primeras unidades cuando se dice mostrar es un sprint review
- Un ambiente de trabajo debe ser abierto – tener pizarrones – el equipo define sus horarios – equipo trabajando cara a cara – el silencio absoluto es un mal signo

HERRAMIENTAS DE SCRUM

Tarjeta de tarea

- Es una tarjeta que cuenta con 5 partes:
 - Código del Ítem Backlog (opcional): identificador del ítem en el Product Backlog.
 - Nombre de la actividad: suele ser una frase verbal que define qué es lo que hay que hacer.
 - Iniciales del responsable de realizar la tarea.
 - Número de día del sprint (opcional)
 - Estimación del tiempo para finalizar la tarea.



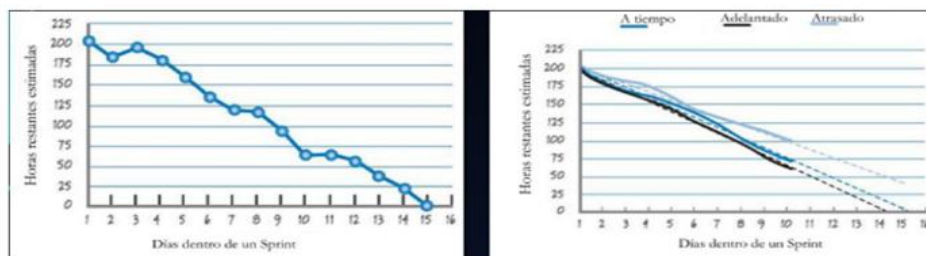
Tablero

- Tablero utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint
- Tiene 5 secciones
 - o **Story:** el nombre de la historia (con estimación en story points).
 - o **To Do:** tareas por hacer en el Sprint (con estimación en horas).
 - o **In Process:** aquí se encuentran las tareas que se están realizando, las tareas se toman del PB (método pull) y entran al SB
 - o **To Verify:** tareas finalizadas que deben verificarse.
 - o **Done:** tareas terminadas: finalizadas y verificadas.
- Tener muchas users con pocos points hace que tengamos muchas cosas pero de granularidad fina, cuando tengo más historias tengo más probabilidad de asignarlas y terminaras antes que termine todo el sprint (es mejor granularidad fina que gruesa ya que en una sprint las cosas están terminadas o no, no importa si estoy al 99%)

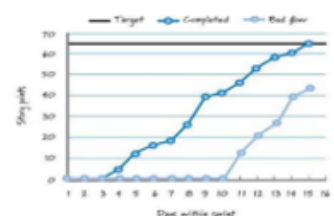
Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... 5 Test the... AC 8 Test the... SC 6 Test the... SC 6
	Code the... 2	Code the... 8	Test the... SC 8		
	Test the... 8	Test the... 4			
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 5 Test the... SC 6 Test the... SC 6
	Code the... 4	Code the... 6			

Gráficos de backlog

- Brindan información acerca del progreso de un Sprint, de una release o del producto.
- Nos permiten pronosticar el progreso
- **Sprint Burn-down chart**
 - o Visualiza con una curva descendente cuanto trabajo queda por terminar
 - o En el eje de las abscisas se coloca el tiempo que va desde q arranco el sprint
 - o En el eje de ordenadas se coloca los puntos de historia a realizar en el sprint
 - o Sumar 8 hs no significa que hice 8 horas de trabajo (gestión de trabajo **vs** gestión de esfuerzo)
 - o Al final de cada sprint los limpio (antes calculo la velocidad para tenerlo de referencia para el prox sprints – no me garantiza misma velocidad pero sirve de referencia para ESTE equipo-)
 - o Sprint de duración muy corto debo actualizar esto bien xq si voy actualizando y veo que no baja significa q voy a terminar el tiempo y no voy a poder cumplir con el compromiso (de ahí puedo ver de corregir algo y si no puedo corregir lo cancelo)



- **Sprint burn-up chart**
 - o Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.
 - o Es un gráfico permanente de todos los sprints



- **Combined burn-chart**
 - o Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.

METRICAS AGILES

- Principios ágiles para elegir una métrica

- Nuestra mayor prioridad es satisfacer al cliente mediante entregas tempranas y continuas de software valioso, funcionando
- El software funcionando es la primera medida de progreso
- La medición es una salida, no una actividad (no tenemos tareas específicas para tomar métricas)
- Medir lo que sea necesario y nada más (lo que agregue valor para el cliente)
- No son extrapolables estas mediciones a otro equipo a otro equipo a otro producto
- Buscamos medir las métricas de producto (no proceso ni proyecto)

- Capacidad del equipo en un sprint

- Métrica que se estima (no se mide)
- Es un compromiso para ver cuanto incremento de producto puede comprometerse un equipo a entregar en un determinado sprint
- Se hace en el Sprint Planning y teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.
- Para equipos más maduros, la capacidad puede ser estimada en Story Points (medida de producto) y para equipos menos maduros estimar en horas ideales (medida de trabajo)
- Estimar en tiempo no es decir un tiempo fijo, en general se trabajan con rangos de horas
- Lo ideal es usar SP ya que si se consumen 5 horas no significa que se terminó una user que llevaba 5 horas pero si se consumieron 5 SP de una user de 5 points significa que se terminó
- Capacidad = Horas de trabajo disponibles por día X días disponibles iteración

- Running teste features (RTF)

- Es la menos usada
- Cuenta la cantidad de features que ya están funcionando
- El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan (por ejemplo pueden estar funcionando 10 user pero no sabemos cuántos puntos tiene cada una de ellas, es una medición absoluta)



Si la curva es constante (llana) o tiene pendiente negativa, entonces indica la existencia de un problema, ya que las características no incrementan o disminuyen en el tiempo. Esta última situación se presenta cuando una funcionalidad del sistema deja de ser válida.

- Velocidad

- Ya la definí antes pero agrego unos conceptos
- Se hace en el sprint review
- Puede tener pocas xq termine muchas pero el PO no las aceptó entonces en la proxima iteración ya están casi todas terminadas y con pequeñas modificaciones las acepta y de repente hice MUCHOS PUNTOS

PLANIFICACIÓN

- Ningún producto de software se puede terminar en un sprint, es necesario entender que hay niveles de planificación diferentes
- **Planificación diaria**
 - o Son las dailys (vamos viendo si en la diaria necesitamos ajuste para cumplir con el compromiso)
- **Planificación iteración**
 - o Planifico una iteración (cada espacio de tiempo que empieza termina y genera un incremento del producto) en scrum la iteración se define sprint
- **Planificación release**
 - o Estimamos cuantos sprints necesito para poder entregar la cantidad de características que definí que integran el release
 - o Tenemos restricciones de presupuesto – tiempo – alcance
 - o Pueden surgir cambios ya que es una estimación
 - o Cada empresa define un release como quiere (un release puede ser cada característica que agregó; un release puede ser lo mismo que un sprint, o puedo soltar 1 release al año)
- **Planificación producto**
 - o Ahora, habiendo estimado cuantos release voy a tener, tengo que estimar cuales van a ser las US que van a ser incluidas en ese release (recordando que el producto backlog puede variar)
 - o Cuanto tiempo y como hago para construir el producto final que yo estimo
 - o Es la sumatoria de los releases
 - o Se siguen teniendo en cuenta el alcance, el tiempo y el presupuesto
- **Planificación portafolio**
 - o Un portafolio es el conjunto de productos de una misma línea (por ejemplo Word, excel, power point para microsoft)
 - o Me debo fijar si me conviene o no invertir en cada producto (a cual le doy más recursos, a cuál menos y cuales discontinuo)

- **Planificación entrega**
 - o Mas a largo plazo y administración del portafolio

Niveles de Planificación

Nivel	Horizonte	Quién	Foco	Entregable
Portafolio	1 año o más	Stakeholders y Product Owners	Administración de un Portafolio de Producto	Backlog de Portafolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de release
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

AMBIENTE DE TRABAJO

- El equipo define sus horarios, pizarrones/postit, abierto (el silencio es un mal signo)

ACLARACIONES

- Scrum no es solo para software
- Los valores de scrum se relacionan mucho con los principios del manifiesto ágil
- Redefinir el PB significa insertar ítems, cambiar tamaño, repriorizar, redefinir, borrar, etc
- Portafolio estimo en talles remera – PB en story points – SB en story point con DoR y puedo dividir en tareas (pero las tareas si tienen horas ideales)
- El product backlog nunca está definido al 100% ni es el ERS (hay cosas q no están xq no se me ocurrieron, xq ya las hizo y no ingresan de nuevo, etc)

ANEXO: AGIL VS TRADICIONAL

TRADICIONAL	AGIL
Cualquier ciclo de vida	Ciclo de vida iterativo incremental
Procesos definidos	Procesos empíricos
Mucho foco en dejar todo documentado	Mas foco en tener software funcionando
Entregas en etapas muy avanzadas	Entregas funcionales recuentes
Seguir un plan muy rígido	Mas orientado a adaptarse al cambio para entregar valor al cliente
Alcance fijo (varia recursos costos)	Recursos y costos fijos (Varia alcance)
Definir todos los requerimientos al principio	Posibilidad de modificarlos según sea necesario mediante retroalimentación
No se priorizan los requerimientos (posibilidad de no satisfacer al cliente)	Uso de product baclokad (prioriza el cliente según le aporte valor de negocio)
Uso de la ERS para obtener los requerimientos	La mejor forma es obtenerlos cara cara
Focalizado en herramientas y proceso	Foco en individuo e interacciones más q herramientas
Duración de cada iteración varia s/recursos y req propuestos	Duración definida (¿con estos recursos q hago en esta sprint definida?
Extrapolable	No extrapolable
Estimar tamaño, esfuerzo, tiempo calendario, costo	Estimar tamaños de las users (complejidad, esfuerzo, incertidumbre)
Estimaciones absolutas	Estimaciones relativas
Estimador experto o gurú	Estiman los desarrolladores
Dogmáticos	Pragmáticos
Estimar una persona	Estimar en grupo
Estimar todo el trabajo	Estimar users que estén por entrar al backload
Estimar todo luego de especificar requerimientos	Estimación al inicio de cada Sprint e ir retroalimentando
Las métricas miden y exponen los avances	El mayor indicador del avance de un producto son los incrementos que se entregan en cada iteración
Métricas para estimar Esfuerzo – tiempo – costos – riesgos	Métricas para estimar Velocidad – capacidad – runing tested features
Método para asignar es Push	Método para asignar es pull (cada persona va y dice che yo me comprometo a hacer x cosa, es una autogestión)

INTRO

- Contrasta el progreso del hardware (algo tangible, construible, que se puede mejorar en su construcción), en comparación con el software (intangible, complejo inherentemente)
- Un proyecto de software puede compararse con la leyenda del hombre lobo, el proyecto empieza normal y familiar, pero que de repente se transforma en un "monstruo" que se sale de control: se come el presupuesto, falla los plazos y genera una frustración aterradora.
- La única forma de detener a un hombre lobo es con una bala de plata.
- La industria del software vive buscando esa "bala de plata": una herramienta, un lenguaje de programación o un método de gestión que, de la noche a la mañana, haga que el software sea productivo, confiable y simple

DIFICULTADES ESENCIALES O INHERENTES

- El software es único en la historia de la ingeniería. Mientras que el hardware ha disfrutado de un progreso explosivo, el software parece avanzar penosamente. Esta diferencia no es falta de esfuerzo, sino una cuestión de naturaleza.
- La "Esencia" del software no es el código que escribimos, sino una intrincada red de conceptos interrelacionados: estructuras de datos, algoritmos y flujos de funciones
- La parte verdaderamente difícil no es "escribir el programa", sino especificarlo, diseñarlo y probarlo conceptualmente.
- Si el problema principal fuera el lenguaje de programación o la velocidad de compilación una herramienta mágica podría solucionarlo. Pero como la dificultad reside en la lógica interna y la especificación de lo que el sistema debe hacer, ninguna tecnología puede eliminar ese esfuerzo intelectual
- Un error de sintaxis es un detalle; un error en la concepción del sistema es lo que mata el proyecto.
- **Complejidad**
 - o El software es más complejo que cualquier otra construcción humana porque, a diferencia de un edificio o un coche, no tiene elementos repetidos.
 - o Al crecer el sistema su complejidad crece de forma exponencial, no lineal.
 - o Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.
- **Conformidad**
 - o Es la complejidad del software que surge porque debe adaptarse a reglas, sistemas e interfaces externas creadas por humanos. A diferencia de la física, donde existen leyes naturales estables, el software depende de procesos de negocio, normas y sistemas heredados que muchas veces son arbitrarios y cambian con el tiempo.
 - o Gran parte de la dificultad no está en programar en sí, sino en hacer que el software encaje con múltiples requisitos externos. Esta complejidad no puede eliminarse con mejores algoritmos o matemáticas, porque viene impuesta desde afuera.

- **Modificabilidad o cambio**

- El software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo.
- El problema es que por esa cualidad los usuarios querrán que haga cosas para las que no fue diseñado originalmente y esto hace que añadamos mas capas de complejidad al software y degrademos su integridad original

- **Invisibilidad**

- El software no tiene forma geométrica, no puedes dibujarlo de manera que se entienda todo el sistema de un vistazo.
- En un edificio, el plano te da una idea clara del espacio, en el software, un diagrama de flujo muestra el control, un diagrama de clases muestra la estructura, pero nada muestra el sistema completo
- Al no poder visualizarlo, nuestra capacidad de razonar sobre el diseño se ve limitada.

DIFICULTADES ACCIDENTALES

- Muchos de los avances más importantes en la historia de la tecnología del software, lograron mejorar significativamente la construcción de sistemas.
- Sin embargo, estos progresos se enfocaron principalmente en resolver dificultades accidentales: problemas derivados de las herramientas o métodos, y no de la naturaleza misma del software.
- **Lenguajes de programación de alto nivel**
 - Permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán
 - Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir
- **Tiempo compartido**
 - Redujo los tiempos de respuesta de un software
 - Permite no desviar el foco de lo que se desea construir
- **Entornos de programación unificados**
 - Facilitan la construcción del software al integrar librerías, formato de archivos, framework, etc

RESOLUCIÓN DE DIFICULTADES ACCIDENTALES → FALSAS BALAS DE PLATA

- Estas soluciones prometen ser balas de plata pero lejos están de serlo
- **Programación orientada a objetos**
 - Permite a los diseñadores del software, abstraerse de tecnicismos para expresar su diseño simplemente en un lenguaje de máquina.
 - Facilita el diseño, pero sólo elimina una dificultad accidental, no elimina la propia dificultad esencial de diseñar software.
- **Inteligencia artificial**
 - Permite resolver muchas cuestiones que reemplazan acciones humanas, pero siempre tiene que haber un equipo de personas que programe y “enseñe” a esa IA cómo actuar. Con lo cual, el factor humano no se reemplaza del todo.

- **Sistemas expertos**
 - o Son sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código, sugerencias en nombrado de variables, etc.
 - o No se quita el factor humano ya que se necesita encontrar expertos y desarrollar técnicas eficientes para extraer lo que ellos saben para destilarlo dentro de bases de reglas de estos sistemas
- **Programación automática**
 - o Se planteó como un reemplazo a los programadores, en el que el usuario describiera sus necesidades y la computadora genere el programa automáticamente
 - o Si podemos especificar el que (requisitos) una herramienta haría el cómo (código)
 - o Esto solo sirve si el programa es algo tan estándar que ya se conocen sus parámetros pero sigue habiendo un problema ante la especificación ya que es extremadamente complicado describir un sistema complejo de forma precisa, completa y sin contradicciones (y si puedes hacer esto ya has hecho el trabajo más duro del software, el acto de generar el código es el paso final y accidental)
- **Programación gráfica**
 - o Propone reemplazar el código textual por representaciones visuales para hacer el desarrollo más intuitivo y reducir errores pero en realidad solo mejora aspectos accidentales del software.
 - o Este enfoque no resuelve la complejidad esencial, que está ligada al problema en sí, sino que simplemente cambia la forma de representarlo. En sistemas grandes, los diagramas pueden volverse tan complejos como el código, sin simplificar realmente el desarrollo.
 - o De hecho, el diagrama de flujo es una abstracción muy pobre de la estructura del software y representa un intento poco efectivo de encontrar un lenguaje de más alto nivel, ya que en la práctica termina en múltiples páginas de cajas conectadas que no sirven como una base sólida para el diseño.
 - o El software es inherentemente multidimensional, por lo que intentar representarlo en un medio visual bidimensional suele hacerlo más confuso en lugar de más claro. Por eso, los diagramas se vuelven ilegibles mucho antes que un código bien estructurado.

RESOLUCIÓN DE DIFICULTADES ESENCIALES → POSIBLES BALAS DE PLATA

- **Comparar antes que construir**
 - o Siempre que sea posible es mejor usar software existente en lugar de desarrollarlo desde cero, esto evita gastar tiempo y esfuerzo en problemas que ya fueron resueltos por otros ya que el costo de replicar software es casi nulo y nos evita recrear una “esencia” ya resuelta
 - o Muchas veces es más económico que el usuario adapte sus procesos al software disponible, en lugar de pagar por un desarrollo completamente a medida
 - o Esta estrategia tiene limitaciones, ya que es difícil de aplicar cuando se tienen necesidades muy específicas o poco comunes, donde no existe una solución previa en el mercado

- **Usar prototipos y refinar requerimientos**

- La dificultad principal del desarrollo no es solo construir el software, sino saber qué se debe construir (ninguna otra parte es tan difícil como establecer los requisitos técnicos necesarios, incluyendo interfaces, personas, máquinas y otros sistemas, ninguna otra parte del trabajo desajusta el sistema resultante si se hace mal ni es tan difícil de rectificar más adelante)
- La extracción iterativa de requerimientos y su posterior refinamiento permiten capturar las verdaderas necesidades del cliente en sucesivas iteraciones, ya que muchas veces los propios usuarios no pueden expresar con claridad todo lo que necesitan desde el inicio. De hecho, es casi imposible definir con precisión todos los requerimientos desde el comienzo.
- El uso de prototipos rápidos resulta clave, ya que permite simular interfaces y funcionalidades principales del sistema, obteniendo feedback temprano del cliente. Esto ayuda a validar ideas, corregir errores y ajustar el rumbo del desarrollo
- Tener un software con pocas funcionalidades pero funcionando es más animador que tener un avance de un software complejo que no funciona

- **Personas capacitadas**

- La calidad del software depende fuertemente de quien lo diseña
- Tener un equipo con buenos diseñadores es tan importante como tener buenos líderes
- Los diseñadores tienen un altísimo valor en el desarrollo de software